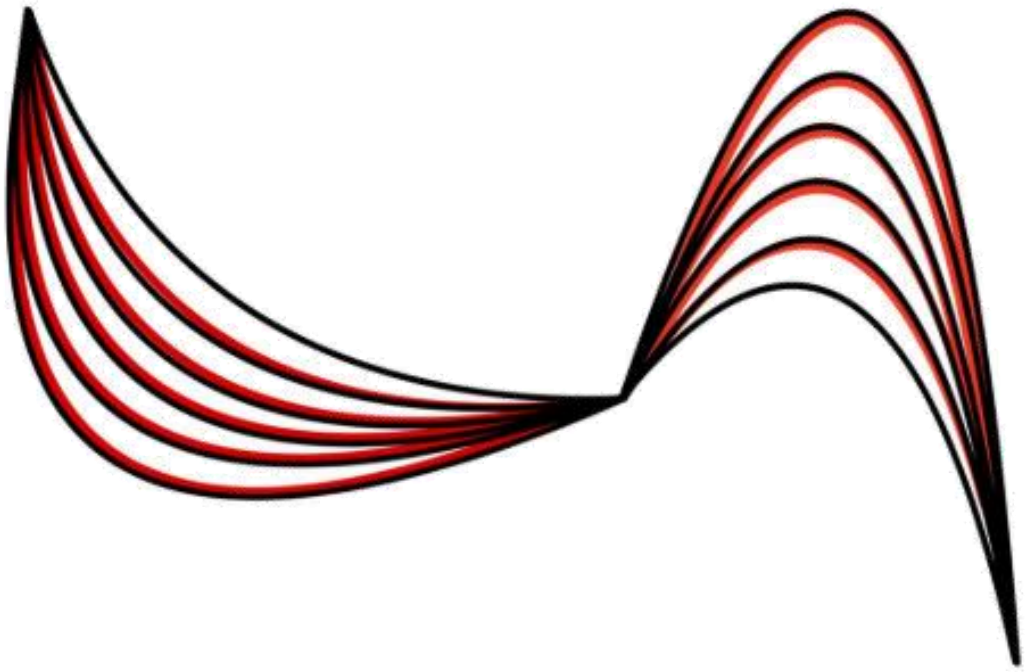
31/05/2022

Universal Testing Machine

‘Waltz’

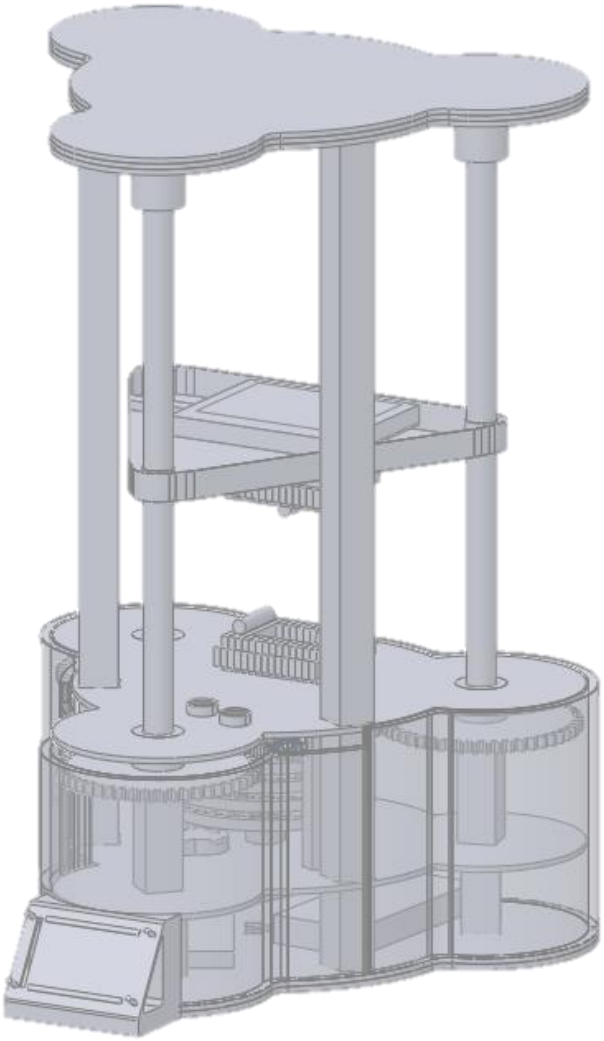
Design Report



Bensile

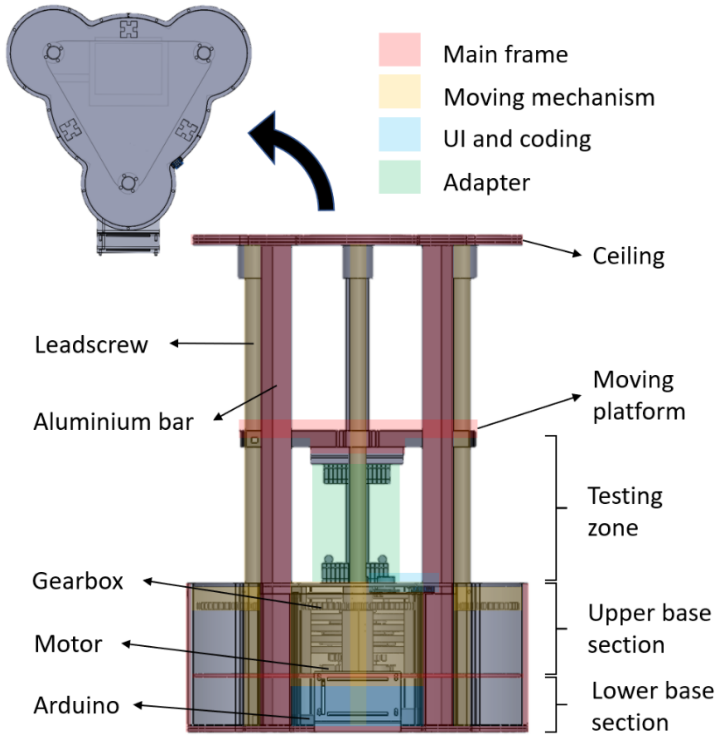
Contents

1. Introduction
2. Subgroup Reports
    a. Main Frame
    b. UI and Coding
    c. Movement Mechanism
    d. Adapters
3. Company Wide Control Tool
4. EDI report
5. Sustainability report
6. Financial report
7. Assessment of project status
8. References
9. Coding Appendix

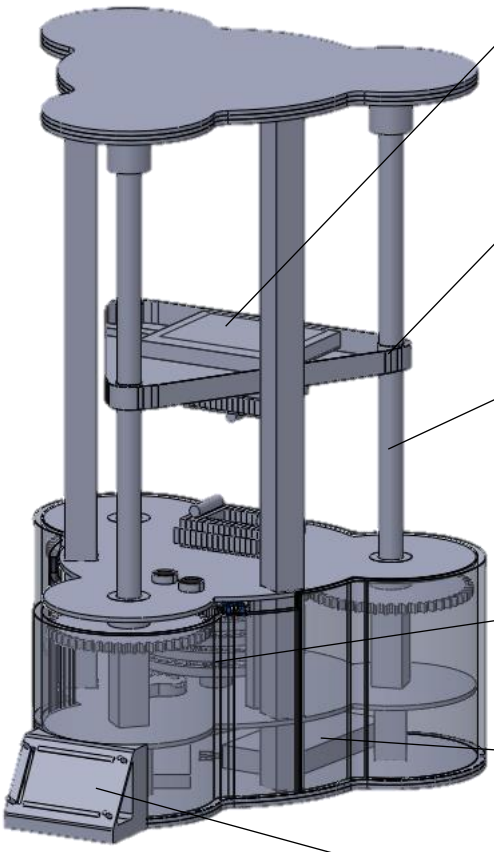


Minjae Kim (he.him) Director	Natalie Ngoh (they.them) Deputy Director	Nora Dopico (she.her) Chief Engineer
Luca Philips-Matteo (he.him) Chief of Automation	Amina Riamah (she.her) 3D Printing Engineer	Amelia Spindler (she.her) Equality, Diversity & Inclusion Officer

Introduction



We have made considerable progress in both mechanical and software aspects, and fabrication of parts has been moving smoothly. There were a few hurdles to overcome, in a financial and manufacturing perspective; however ultimately, we prevailed, and we managed to produce a functional, sustainable, and polished machine. We are excited to present our final design – affectionately named Waltz.



The moving platform houses a loadcell and a tensile or bending test adapter can be mounted onto it. After fabricating adapters using various techniques, laser-cut

We planned to fabricate leadscrew nuts by letting epoxy set around the leadscrews. However, it adhered too tightly so we 3D printed them instead. This brings problems with shrinkage which we discuss later.

The equilateral leadscrew placement provides stability. However, using three leadscrews instead of two reduces the range of the moving platform. Hence, we switched the gearbox to the upper base section and elevated the leadscrews with aluminium bars.

The gearbox multiplies the output torque exerted by the motors, which we found to be weaker than expected during the testing phase.

The Arduino is wired through fitted holes towards the mains and the upper sections. Wires are concealed and protected as they are embedded in the aluminium bars.

Waltz's clover shaped frame is designed to minimise the material used. There is a display which allows the user to customise their test variables and show data.

3
Subgroup Reports
Main Frame
Prologue

As the subgroup leader in the main frame subgroup, Amina Riamah was responsible for creating the CAD of the middle moving platform and writing this report. In the prototype phase, she developed the square base of the machine as well as the protective cover for the machine. However, as the design progressed, responsibility for the base was handed to the company director, Minjae Kim and deputy chief engineer, Aaron Edobor. Utilising technical drawings of the load cell compartment by chief of automation, Luca Philips-Mateo, Amina CAD and produced simulations for the moving part complementary to the finalised design: Waltz. Lastly, as subgroup leader Amina assured communication between the main frame and adapter subgroup, led by Amelia Spindler, in order to coordinate design progressions.

As the deputy chief engineer, Aaron Edobor was responsible for making several constituent parts including the base plate, middle plate and static top plate. Following the prototype phase, he was involved in developing a circular design: Lorelai, which comprised elements of the initial square base design in a new concept drawing constructed by Nora Dopico and Minjae Kim. The development of Lorelai led to the finalised design of Waltz, in which Minjae further CAD and revised the designs into manufacturable parts. Alongside work on structure of the machine, Aaron, Amina and Minjae helped Nora record parts used in both final and prototype products.

Minjae Kim as previously mentioned, used the previous CAD drafts that Aaron produced and created manufacturable CADs, such as adding slots for sensors, joints to connect the parts and new parts such as a display stand for the TFT display. As the director, although he was mainly associated with the main frame subgroup, he gave suggestions to other subgroups and facilitated communication and cohesiveness throughout the company along with the deputy director. At the start of term, he presented a general roadmap of the term to give an overview to the company members. Furthermore, he was heavily involved in the manufacturing aspects of Waltz, by communicating effectively with the laser cutting/3D printing engineers. To add, he communicated wit

Su Peng effectively executed his role within the subgroup itself aswell by balancing the structural integrity of the machine with readily available materials. He researched and ordered materials such as the PMMA used to produce parts of the base, and occasionally assisting Amina and Aaron with making simple CAD designs and laser cutting software.

Minjae Kim (he.him) Director	Aaron Edobor (he.him) Deputy Chief Engineer	Su Peng Wang (he.him) Chief Financial Officer	Amina Riamah (she.her) 3D Printing Engineer
		Su Peng Wang	

Content

The main frame group was responsible in producing the physical exterior of the machine which integrates every element of Waltz: the moving mechanism, the adapters and the UI and coding. Each part of the machine has been separated into distinct files to facilitate the development of design and ease of assembly.

In the prototype phase, we designed the base to incorporate an upper and lower section divided by a middle plate supported by pillars. However, we met several issues such as the size of the central gears requiring increased space as well as the need for stronger supports to prevent the middle plate from deflecting. The originally PLA 3D printed support pillars were replaced with stronger aluminium bars to ensure a strong basal foundation. We also added a circular extruded cut in the middle of the ceiling to save material to be reused for cutting gears for the improved gearbox. The user screen was initially located in the specimen testing zone, however upon reconsidering the safety aspect of it, we optimised a 60º viewing angle from an extruded section of the base. The moving plate involved difficulties from its design to the spatial limitations of the 3D printing bed, and several designs were put forward to address the different functional aspects of it.

During the production stage, several observations were made about the viability of 3D printing and laser cutting the main frame. Firstly, three versions of the loadcell box were designed, and the chosen design was produced multiple times with varying production settings. It was found that whilst 40% infill produced a solid finish, 30% infill was more time and material efficient. Unexpected expansion in lieu of planned shrinkage caused many revisions to be made for the loadcell box, but we minimised this by reusing waste PMMA trimmings to fill out the uneven gaps between the 3D printed surface and the irregularly shaped NFC sensors.

Base and ceiling frames

3D printed wells secured to the top plates allow the leadscrews to remain upright while still being able to freely rotate. The two top plates are connected via a glue adhesive. The base middle plate divides the base into a lower section – housing the UI and code – and an upper section – housing the gearbox. The base top plate acts as a lid for the base of the machine and it will be connected to the adapter's female part. Both the top and bottom basal plates fit snugly into the curved 3D printed walls of the base and are supported by aluminium pillars that rest on the lowermost plate.

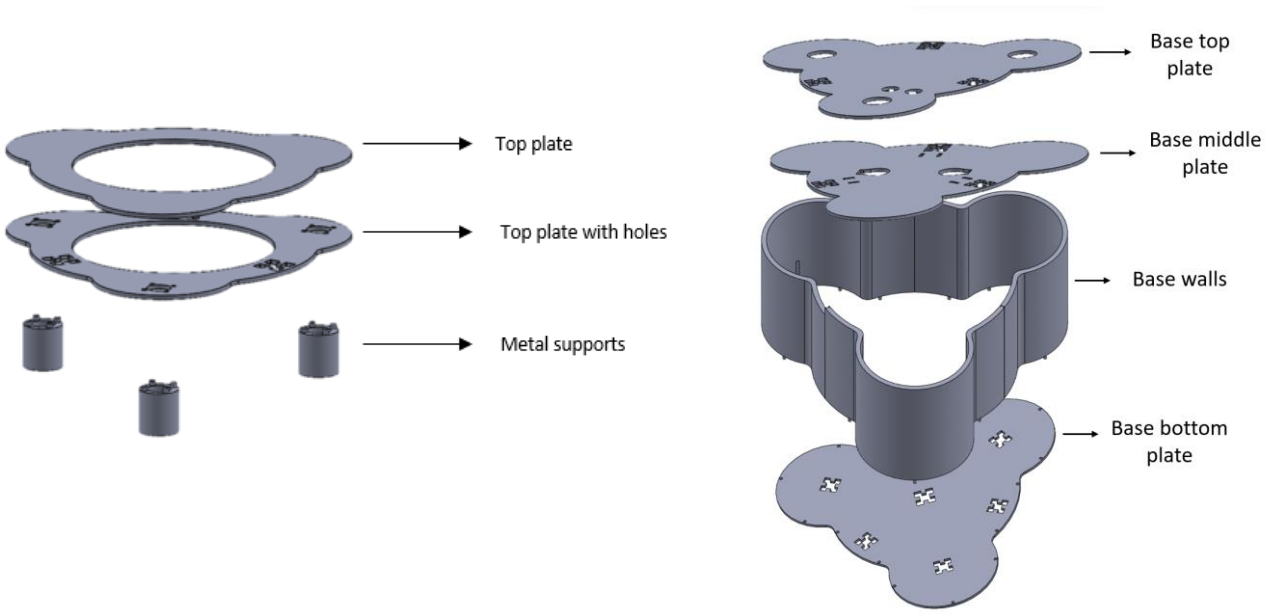


Figure 1: Exploded view schematic of ceiling plates (left) and basal plates (right).

Moving platform

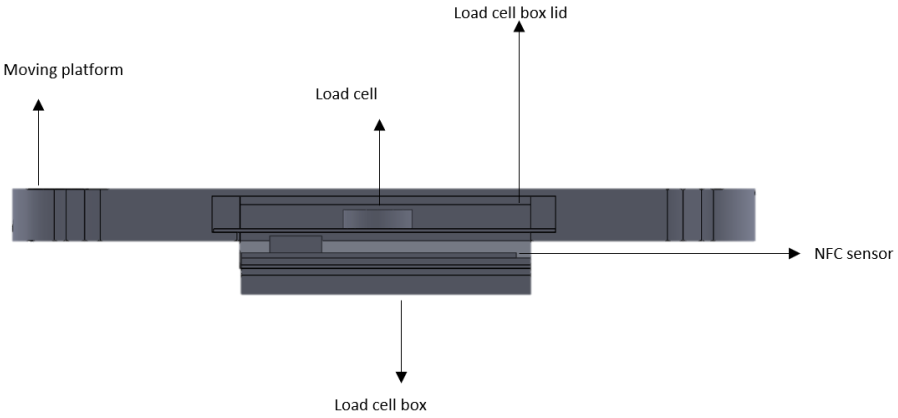


Figure 2: Schematic of moving platform (front view).

Internal threading of the holes connects the moving platform to the leadscrews, converting the leadscrew's rotation into vertical motion of the platform. The moving plate is comprised of a couple of external components: the loadcell box, containing the loadcell and NFC sensor, and the lid. The loadcell rests on a platform beam which in turn rests on extruded ledges in the moving platform. The adaptor is attached to the bottom of the loadcell box via screws. The NFC sensor detects which adapter is attached (for bending or tensile testing) according to the respective NFC sticker.

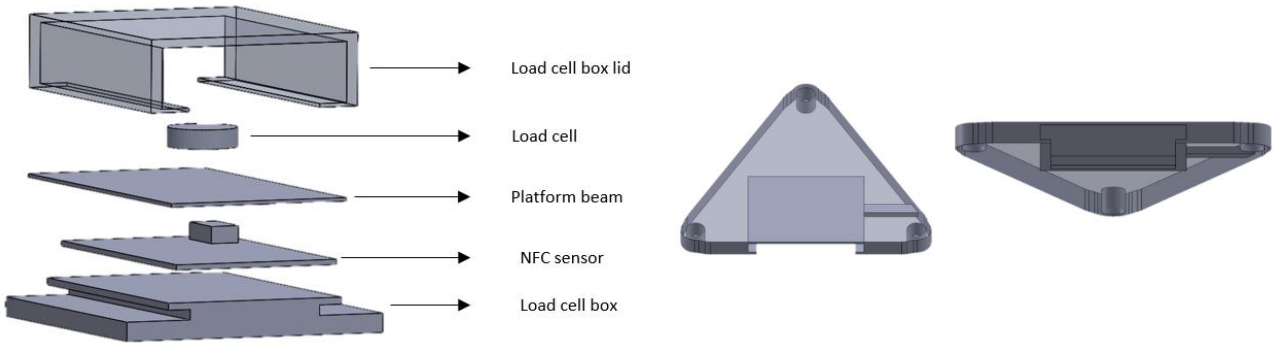


Figure 3: Exploded view of loadcell box components (left) and views of moving platform (right).

The loadcell has an extruded button which is in constant contact with the loadcell box lid. When a force is applied, the button deforms and the load it experiences is transformed into an electrical signal. An extrusion in the middle plate allows the loadcell to be wired up to the Arduino located in the base. The wires are concealed in the grooved of the metal supports that run from the ceiling to the base of the machine. Recycled PMMA cuttings were repurposed as support pillars for platform beam and the NFC sensor which have uneven surfaces.

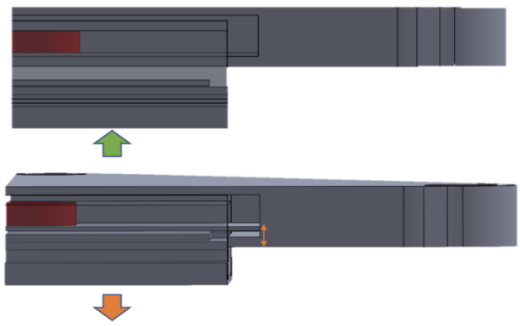


Figure 4: Loadcell in compression (top) vs in tension (bottom).

During a bending test, the loadcell box is displaced upwards, pushing the platform beam upwards, resulting in the loadcell getting compressed upwards against the lid. During a tensile test, the loadcell box is pulled downwards, so the lid moves downwards as well since the loadcell box has two ledges that rest on the ledges of the lid. In this case, the platform beam remains stationary and the loadcell gets compressed by the downwards force applied by the lid.

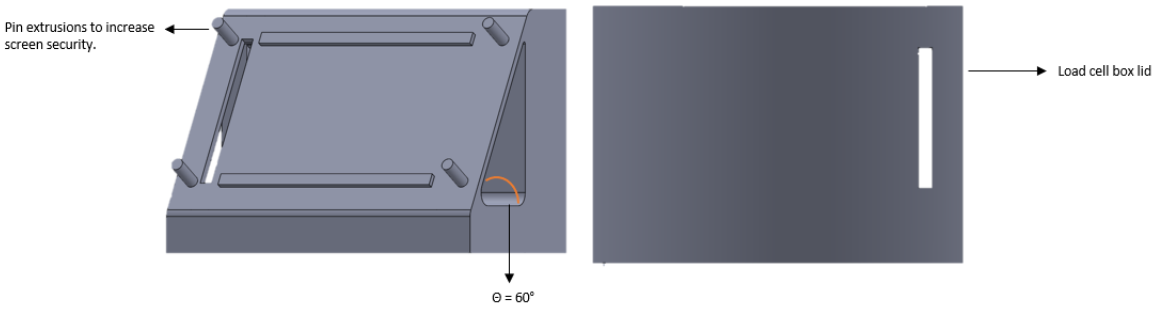


Figure 5: Schematic of LCD screen mount front view (left) and back view (right).

The extruded LCD mount optimises the viewing angle (60º) for the user whilst minimising material. Pin extrusions indicate the mounting positions to secure the screen while the hole allows the screen to be wired up to the Arduino in the base.

7

UI and Coding

Prologue

As Chief of Automation, Luca was responsible for the leading the development of the code providing new ideas, such as applying RFID technology for a smoother user experience and communicating with other members in the company. Safeeyah Ahmed lead development of data collection and overall operation of the machine. Konstantin Merabishvili contributed to developing confirmation code and data analysis. Moreover, programming and applying the ultrasonic sensor that is used for measurement detection.

The UI and Coding subgroup played an essential role in the function of Waltz. The main role of this subgroup was to provide the interface through which the user will interact with, taking measurements, such as cross-sectional area and beam bending cross-sectional shape as the main inputs from the user.

Luca Philips-Matteo (he.him) Chief of Automation & Control	Konstantin Merabishvili (he.him) 3D Printing Engineer	Safeeyah Ahmed (she.her) Chief of Sustainability

Content

The numerical values are received through a keypad located on a 2.4” touchscreen display, which the system will store to aid in the calculation of stress, strain and Young’s modulus values during a test. Cross sectional shapes will be inputted through intuitive buttons on the display, each with a drawing of the respective shape (Figure 1.d). Other interactions include the confirmation of the type of test to be carried out and viewing data. Initial designs were to use a separate number pad, but after further analysis, we decided to remove the number pad allowing for the same functioning in a more compact interface, due to the versatility of a touch screen. The type of test to be carried out will be selected through the placement of the interchangeable test components on to the machine. RFID sensors at the top and bottom of the machine will detect the NFC tag inserted into each component, making sure no mistakes are made in the process. The user is then prompted to insert the sample, if a tensile test is to be carried out the ultrasonic sensor will measure the length of the sample. The test will be carried out upon the confirmation of the user, with a clear page (Figure 2.f). Temperature and humidity of the environment will be recorded from a sensor before the test starts in the Arduino and displayed in the results page.

As the test is being carried out, more inputs will be received, particularly the force exerted on the sample and the change in length, both measured every 500ms. These will be stored in the Arduino for analysis after the test. The machine will have a failsafe incorporated into the ultrasonic measurements, so that the machine does not damage itself. This failsafe will be a maximum and minimum measurement that can be received from the ultrasonic sensor or once the machine can no longer move. Once this is achieved, the test will be concluded, and analysis will begin. Our initial idea was to use the movement of the stepper motors themselves, but we found that this may not give an accurate length measurement, and so it was decided that ultrasonic sensors would be more accurate and therefore more suitable. The main outputs of the code will be the mechanical properties of the samples being tested. The tensile test and Bending test will both output the UTS, Young’s Modulus and Maximum strain of the sample. These will all be calculated through the code illustrated in Figure 3 and outputted on to the respective page on the display (Figure 2.h.). The user will then be given the option to begin a new test, upon which the machine will reset itself.

The main changes that have been made are a smaller display, 2.4” as opposed to 3.2” and the removal of the push button, cancelling the test will now be completed by interactions with the screen. The main inclusion is the ultrasonic sensor, for measuring distance and the use of stepper motors that allow for better control of the machine.

As referenced in the main frame section, the Arduino MEGO 2560 is in the base of the machine, with a small outlet. The whole system is powered by a power supply component connected to a breadboard. Two 5V stepper motors, a 2.4” TFT display, an ultrasonic sensor, a RFID module and temperature and humidity sensor is in the base. Wiring that uses male-female jumper wires that run to the moving platform of the machine allows us to connect the load cell and a RFID module. These are used to detect the top component attached to the machine and the load cell located in the load cell box.

For the machine to accommodate all the electrical components, the main frame had to be altered so that a compartment in the bottom of the machine could be available. The load cell also proved a challenge, since it had to be designed to measure both tensile force and compressive force.

List of components:

- Arduino MEGA 2560
- 2 x 30 port breadboard
- 2 x L293D transistor
- 2 x MFRC – 522 RFID
- DHT11 Temperature and Humidity sensor
- HC-SR04 ultrasonic sensor
- 2 x 28BYJ-48 Stepper motor
- TE Connectivity FX292X-100A-0100-L Load Cell
- 2.4” 4D Systems uLCD-24PT Display

The code has not been completed yet, however all the required functions for the machine to work successfully are present. Appendix 1 shows the UI structure of the code, dealing with the interaction of the user with the machine. The user navigates through the forms previously described, inputting numerical values into keypads. Since a display is sued, all interactions are dealt with the myGenieEventHandler function, called by genie.doEvents(). The function will dequeue an event and handle the event. This code is associated with the company wide control. A sample of the code is shown in Figure 6.

```
if(Form == 2 or Form == 4){                    //this is essentially a keypad, as the built in one did not seem to be working so I have made my own rudimental one.
  Serial.println(Shape);
  Serial.println("Form 2 or 4 detected");
  if(genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_WINBUTTON, 31) or genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_WINBUTTON, 3)){ //The enter button, storage of the number entered.
    if(Form == 2){
      TArea = flt5;
      str = "Value stored";
      genie.WriteStr(2, str);       //change string displayed
      delay(1000);
      genie.WriteObject(GENIE_OBJ_FORM, 5, 0);   //change to form 5
      delay(200);
      Serial.println("Next form for both 2 and 4");
      Form = 5;
      decimal = 0;
      return;
    }
```

Figure 6: A sample of code associated with inputting values.

All the commands will be filtered by the form in which the display is in, stored in a variable. Once it is filtered, genie.EventIs will check the button selected, as each one has a unique identification code. For example, here the enter button on one of the 2 keypads was selected, shown by the user pressing winbutton 3 1 or 3. Subsequently, the value will be stored, with a storage confirmation message will appear on the display for 1 second before the code moves on and changes to the next form. This example is for inputting the cross-sectional area of a sample for the tensile test.

The code shown in Appendix 1 will call other functions, including the use of the RFID modules to determine the male part added, and therefore the test to be conducted while the first form is activated, as in Figure 1 of the company wide control tool section. Other functions to be called during the machine use is that of the temperature and humidity sensor, which will be called to store temperature and humidity of the surrounding environment on start-up, at the point the selected test is activated and once the test has been completed. These will be taken as inputs from the DHT11 sensor. The most important functions to be called are the test functions, which will conduct the selected test upon calling and taking in inputs from the ultrasonic sensor and the load cell. Once the tests have been carried out, a function for data analysis will be called to find the mechanical properties of the material, such as the Young’s

Modulus or the ultimate tensile strain. A summary of the way in which these functions work is shown below. All functions are contained within Appendix 2.

```
    for (byte i = 0; i < nfc2.uid.size; ++i) {



 Serial.print(nfc2.uid.uidByte[i] < 0x10 ? " 0" : " ");
 Serial.print(nfc2.uid.uidByte[i], HEX);
 type2.concat(String(nfc2.uid.uidByte[i] < 0x10 ? " 0" : " "));
 type2.concat(String(nfc2.uid.uidByte[i], HEX));        //hex UID taken from the NFC tag are converted into a string

      }
Serial.println();
Serial.print("Message : ");
type2.toUpperCase();
  if (type2.substring(1) == "13 87 0C 0C") //UID of detected NFC is compared to known UID of the male components
{
  Serial.println("Tensile 1");
  Serial.println();
  delay(300);
}
```

Figure 7: Code when RFID sensors when an NFC tag is detected.

The UID of the tag is read and stored, before being compared to the UID of the male components, as can be seen in Figure 7. This will then store whether the tag is valid, and what male component it corresponds to if valid. This will be an output of the function for further analysis in the main code.

The data analysis function, shown in Appendix 2 with example values, will find the maximum stress and strain values and take the gradient over a linear range of the data to find the Young's modulus value.

The functions for the tensile and bending processes work by taking inputs from the screen that the user types in using the LCD screen. These include if the sample is a cylindrical shape or a cubic shape. The code will then calculate the radius, if necessary, and the cross-sectional area of the sample, which is used for calculations of stress along with values of load which are found using data from the load cell. With the use of an ultrasonic sensor to determine the differences in length of the sample, the strain values can also be calculated for the tensile test. For the bending test, values of stress calculated depend on not only the shape of the sample and load, but also the distance between the two supports on the plate. This is a value that is stored as a constant in the code along with the value for the height of the machine. The code also allows for values of temperature and humidity to be found using sensors. All values of stress, strain temperature and humidity are stored in separate arrays which are available for use during data analysis. The ode is currently written to terminate the test when values of load across an average of 5 readings decrease drastically compared to the average set of the machine height minus 0.5cm. The results will then be outputted on the display by the UI code, giving both engineering and true values for stress and strain.

10
Movement Mechanism

Prologue

The movement mechanism subgroup is responsible to create the gearbox that connects to the motors and transfer the torque to the leadscrews, making the moving plate able to lift or go down, therefore creates the origin of tensile and bending test.

As the subgroup leader, Wei-Jung Chen was responsible for writing this subgroup report and helped CAD the motor gears in the initial design. In addition, he helped make the epoxy resin for leadscrew threads in the lab. As chief of production, he made a spreadsheet for production record, which recorded all the parts that were produced through 3D printing or laser cutting. The PIDs, dates, manufacturing methods, settings of the machine and feedbacks are clearly recorded onto that spreadsheet from the prototype stage forward. Besides that, he wrote about “lessons learnt” throughout the production period for future students to reference. He also drew the mechanical drawing of the gears that were involved in the mechanism.

As the chief engineer in the company, Nora Dopico compiled all the parts and their respective versions in a parts list, which consisted of recording UID’s, measurements, designers, dates, and materials, among others. Additionally, she wrote the risk assessment and gave pointers to other group members, such as the sustainability officer, on crucial design edits that were made to improve the projects’ environmental implications, specifically referring to the user-machine interaction. She also CAD and produced technical drawings for the initial central gear in the movement mechanism and resolved issues regarding said mechanism in her subgroup. Furthermore, she made and communicated fundamental changes to the functional and aesthetic redesign of the machine to the relevant subgroups of the company, such as switching from a 4-rod design to a 3-rod design.

As the deputy director in this company, Natalie Ngoh overlooked the progress of the various subgroups and made sure that the company was moving in the correct direction. In particular, the mechanical aspect of the machine had to be coordinated between the movement mechanism, adapter and main frame subgroups to ensure that all the parts fit well together in the final assembly of the machine. They also collated reports from each subgroup to form the final design report, ensuring that the entire report was cohesive and that there were clear links between every part that each subgroup contributed towards. As a member of the movement mechanism subgroup, they made calculations and designed the gear system using CAD to include a series of planetary gears, as well as secondary gears that were used for power input and output from the gearbox. Natalie also supervised the fabrication of 3D printed and laser cut parts for the movement mechanism subgroup during the testing phase and adjusted in preparation of the final design phase.

Natalie Ngoh (they.them) Deputy director	Nora Dopico (she.her) Chief Engineer	Allen Chen (he.him) Chief of Production

Content

In the prototype stage, we designed the central gear-motor gear coupling, which looked promising. However, we failed to analyse and calculate the torque needed for the gears to break the specimen during the test. Our initial response was to replace our DC motors with stepper motors since stepper motors have a larger torque. One drawback is that stepper motors move in small discontinuous steps, which could affect the smoothness of the stress-strain curve obtained. However, this can be mitigated by reducing the time interval of steps by Arduino. The torque was still insufficient after changing motors, so we redesigned the gear box with the aim to increase the torque until it is enough for tensile or bending tests. During the redesign process, we paid greater attention to the gear teeth ratio, and found that a gear ratio of 6 is the most ideal for increasing torque while having our gears remain at reasonable diameters.

During the production stage, we cut the leadscrews into three parts that have equal lengths by rotary saw and drilled the holes on each one. For the gears, we chose PMMA as our material for gearbox because it is strong enough and relatively cheap compared to metals. We used laser cutting to fabricate all our gears as this was the most precise method. For the chassis of the gearbox, we decided to 3D print them as they needed to be custom made. One difficulty we faced was that 3D printed parts tend to shrink and warp, which affected the fit of the parts. Instead of purchasing leadscrew nuts, Natalie and Wei-jung attempted to make them using epoxy resin in the lab. Unfortunately, it is possible that we did not brush the leadscrew with sufficient oil, leading to the epoxy getting stuck to the leadscrews. While we were trying to take it out, we also found that the specific type of epoxy used was too brittle, so we decided not to redo the experiment and try to 3D print the leadscrew nuts instead.

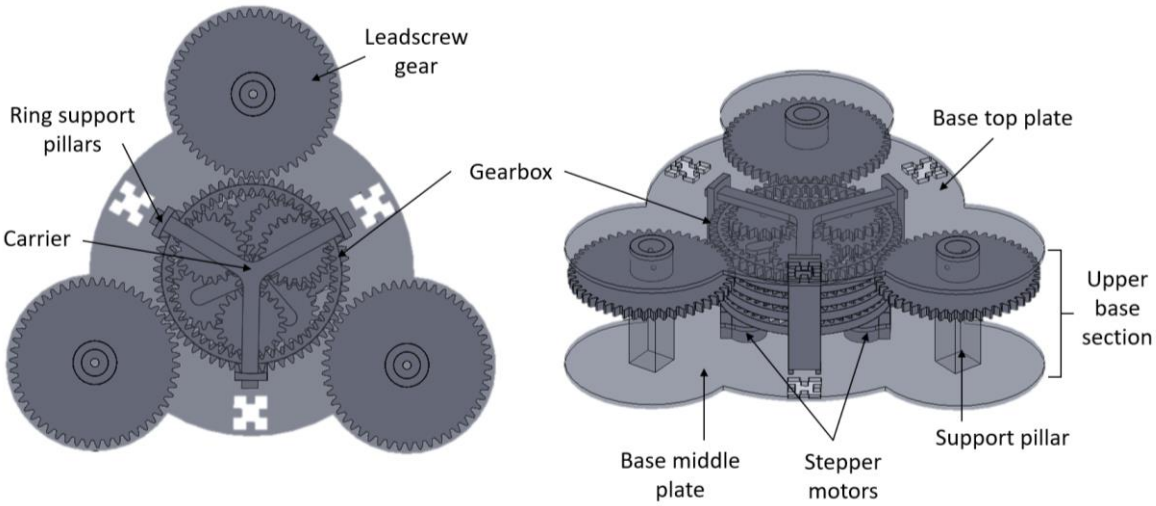


Figure 8: Schematic of planetary gearbox system, with top view (left) and isometric (right).

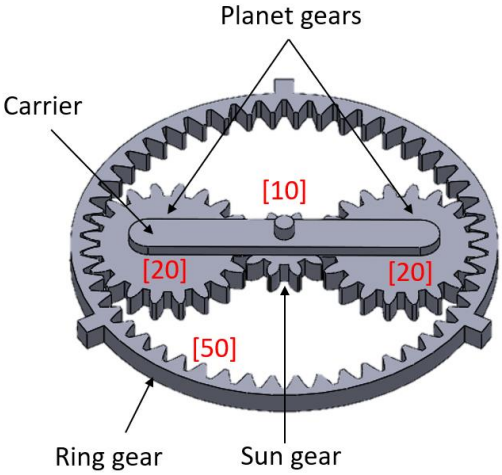
Motor

<u>Stepper motors</u>
Our machine is powered by a double stepper motor set-up, and power is combined from the motors to a single gearbox using a series of spur gears.

<u>Bearing</u>
A bearing was used to reduce the amount of friction between the very bottom gears – which have the highest rpm – and the base middle plate, which would reduce energy wastage and unwanted heat generation. This also protects the electrical circuit directly below the gear box.

Gearbox

<u>Planetary Gears</u>
The gearbox is a mechanical system that takes our input torque, scales it up, and outputs it. The torque follows the law of gearing (Eqn, 1), showing a dependence on gear ratio. We used planetary gears (Figure 8), which consists of a central sun gear, two planet gears that orbit around it, and a ring gear. Each planetary gear takes an input at the sun gear, steps up the torque by a factor of 6, and outputs it through the rotation of the carrier. The carrier is attached to another sun gear and this process is repeated in the next planetary gear above. We have a total of 5 planetary gears in series, which means that this gearbox can increase the torque from our motors $6^5 = 7776$ times. Another consequence about doing so is that the angular frequency of the output rotation will be greatly slowed, which prolongs the length of the test. This is useful for testing samples under quasi-static conditions.



Fixed ring gear

$$\frac{\tau_{out}}{\tau_{in}} = 1 + \frac{z_r}{z_s} \qquad \text{Eqn. 1}$$

Fixed carrier

$$\frac{\tau_{out}}{\tau_{in}} = \frac{z_r}{z_s} \qquad \text{Eqn. 2}$$

τ: torque
z_s: number of teeth of sun gear
z_r: number of teeth of ring gear

Figure 9: Schematic of an isolated planetary gear with the number of teeth for component gears shown in red, alongside derivations from the Willis equation [1] for planetary gears in fixed ring and fixed carrier configurations.

Ring support pillars

There are three ring support pillars that hold four ring gears the topmost carrier in stationary. The support pillars are attached to the gears and the base middle plate by using extrusions that fit together, removing the need for any adhesives or screws used.

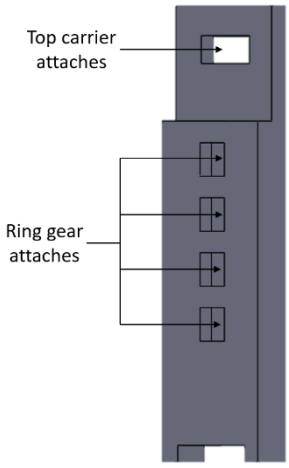


Figure 10: Schematic of ring support pillars.

Transition / hybrid gear

The topmost ring gear differs from the rest in that it has a set of external teeth and internal teeth. In this layer, the input remains as the sun gear, while the output is now the ring gear instead of the carrier.

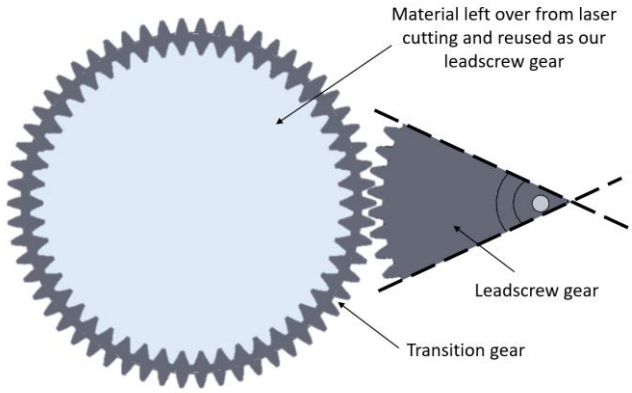


Figure 11: Schematic of transition gear (left) and a section of leadscrew gear (right).

Leadscrew gear
The leadscrew gear consists of a 50-tooth spur gear which is secured to a leadscrew support. This goes onto one end of a leadscrew and is fastened to it by inserting a rivet through the holes in the side of the leadscrew support and drilled through the leadscrew.

Leadscrews

The three leadscrews play a pivotal role in the mechanics of the machine as it converts rotation about an axis to translation along that same axis. This allows us to control the up-and-down motion of the moving platform which will strain the sample.

Leadscrew nut
The leadscrew nut is a small component; however, it needs to be extremely precise in order to match the thread of the leadscrew. Instead of purchasing three leadscrew nuts using majority of our budget, we decided to fabricate it. Initially, we used epoxy which we poured as a liquid and let set. However, the epoxy adhered to the leadscrew instead of spinning around it freely. We then decided to go with an alternative fabrication method of 3D printing the leadscrew nut as PLA is equally strong as epoxy and thus is a good substitute.

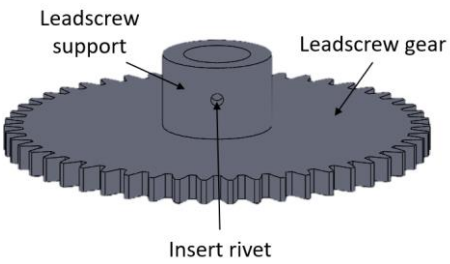


Figure 12: Schematic of leadscrew gear (bottom) and leadscrew nut (top).

Adapters

Prologue

In the first prototype, we decided that the height of the adapters required reducing to maximise the working height for bending or tensile tests. Thus, we redesigned the male part to have one dove tail joint rather than two, reducing the height by more than 42%. Within the final design, we also implemented a gap for an NFC-RFID tag/detector to be placed in the female part and the male part. Furthermore, we contributed to designing the load cell box so they would attach easily.

Amelia Spindler, the subgroup leader was responsible for writing the adapter subgroup report and monitoring the members of the subgroup. Since she was also the Equality Diversity and Inclusion officer, she sent out a weekly questionnaire to monitor key aspects of the company such as, communication, workload and individual issues. Moreover, when problems arise, she spoke to the relevant people and the Director team (Minjae and Natalie) to solve them as soon as possible. Furthermore, she aided in designing the adapter which included multiple sketches and redesigns and constructed the finalised adapters found in Waltz.

Marco Marabese and William He are both Laser Cutting Engineers within the company, assisting all mechanical subgroups since the establishment of the company, which ranged from processing MDF, PMMA and cardboard. In the adapter subgroup specifically, Marco was deeply involved in printing parts; printing over 60 cardboard parts for the prototype and 40 MDF parts for the final piece. In addition to this, they both contributed to the CAD throughout the project and assemblies.

Amelia Spindler (she.her) Equality, Diversity & Inclusion Officer	Marco Marabese (he.him) Laser Cutting Engineer	William He (he.him) Laser Cutting Engineer

Content

Bending

When designing the bending part, we ended up only delivering three point bending rather than both the three and four because the UI team found that there were too many unknown variables for a four point bending test. Moreover, a gap in the bottom male bending part was added, which allowed greater displacement for the sample, without compromising on the height of the machine itself.

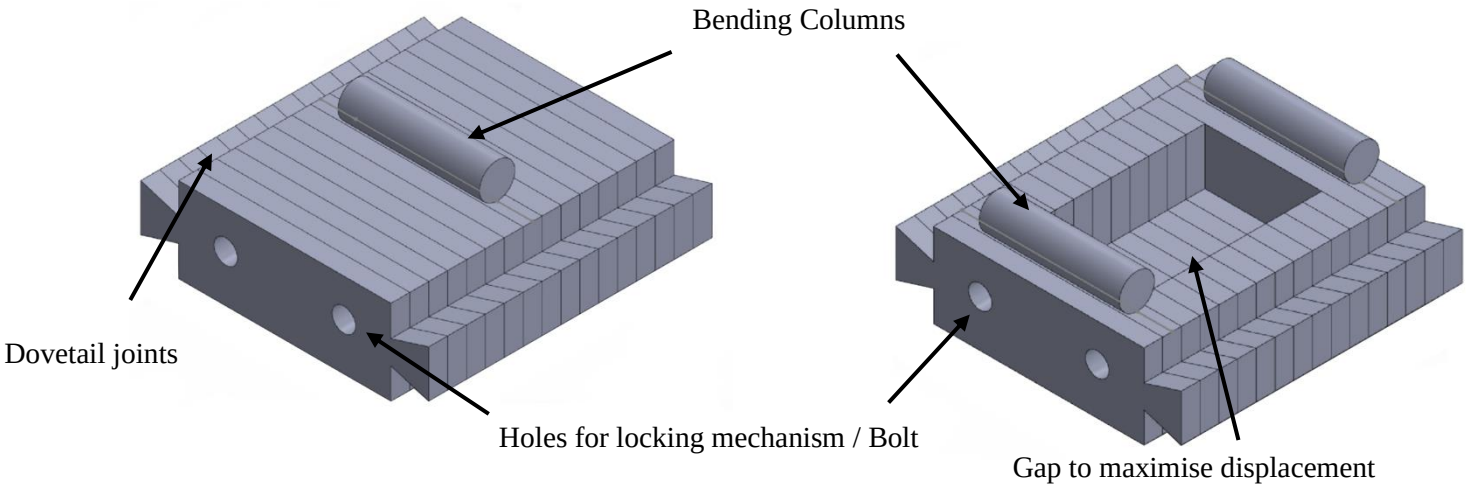


Figure 13: Bending top and bottom male adapters are shown.

Tensile

The fundamental design of the tensile male adapter stayed the same throughout the design process. The support rod and the grips were made from PLA and the tensile tightening rod is made from a bolt attached to the grips. To increase the grip, we are using rubberised tape to secure the tightening rod.

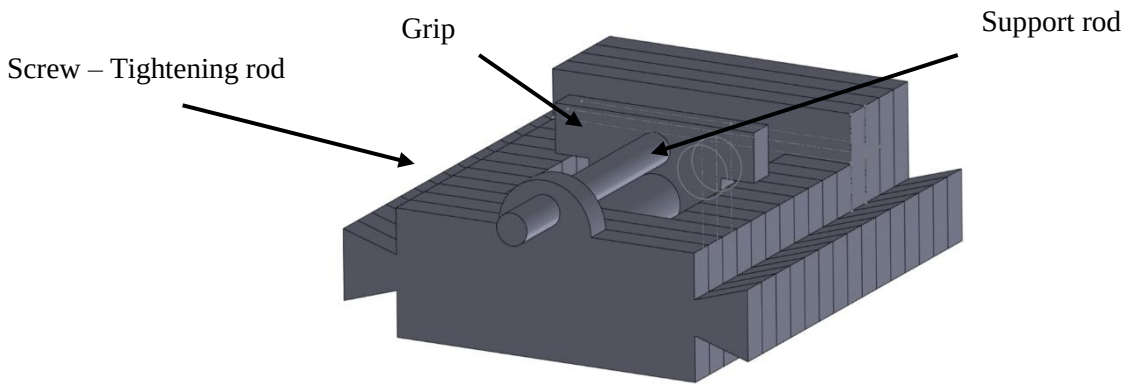


Figure 14: Schematic of tensile male adapter.

Locking Mechanism

The locking mechanism is the same for both bending and tensile, we originally wanted sex bolts, however they were too expensive, and the department did not have them. Therefore, we made a compromise, to use standard nuts and bolts but in turn, making the adapter changing less smooth.

Material

In the development of the final parts, we did a test run using additive manufacturing and printed a mock female-male pair. However, we found that it was difficult to get the correct measurements for the male part due to its shrinking nature and we would have had to make many iterations to get the right fit. Additionally, it would have also made the incorporation of the locking mechanism very difficult therefore, deciding that we would carry on with our original laser cutting method. MDF was also found to be more sustainably attractive material which we are laser cutting from.

When laser cutting the MDF thickness of 3mm, the settings chosen only permeated part way though the wood, resulting in over half of the wood becoming unusable. Fortunately, we still had enough wood left to cut the pieces, since Amelia realised that parts of the male could be placed on the inside of the female parts to save area.

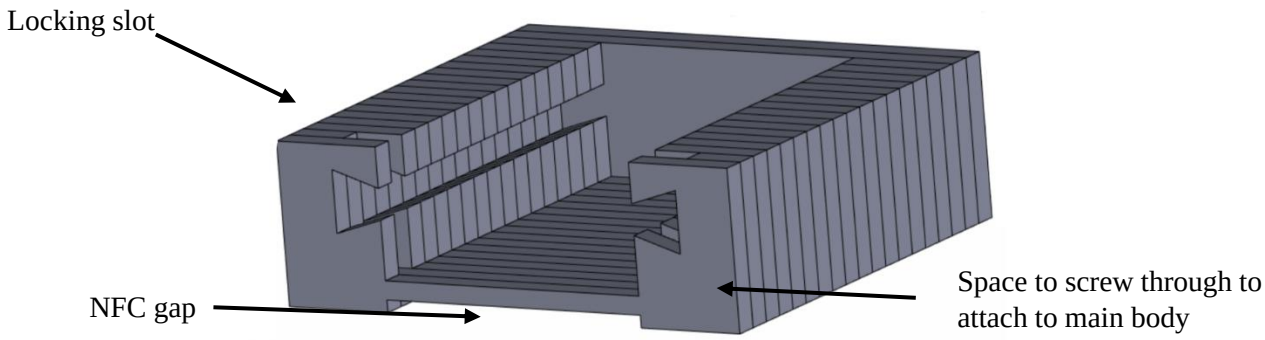


Figure 15: Schematic of female adapter.

Our next step regarding manufacturing of the part is to finalise the dimension of the 3D printed parts. We have already printed the first round of parts but need to see how they fit with the laser cut parts and edit the dimensions accordingly or post process them (sanding) to work.

Setting up the Plant

Before every step, disconnect the power supply.

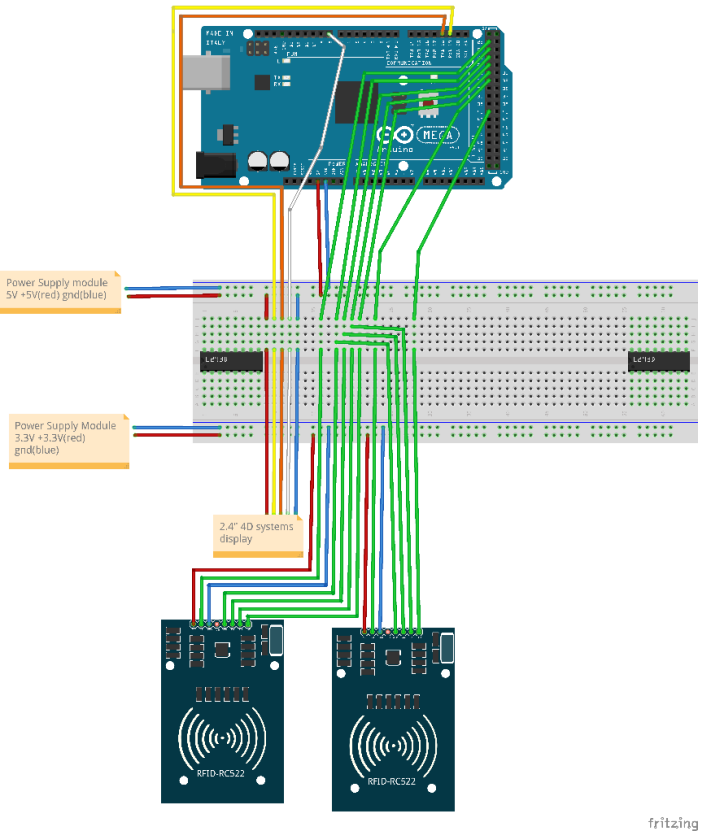


Figure 16

To set up the plant, begin by connecting one power supply module to a bread board as shown above, with the jumpers being at 5V and 3.3V. Connect the Arduino to the power supply by running a wire to the 5V pin and ground pin, plug in the power supply and rest by observing whether the power light lights up. Add a transistor to each side of the breadboard, ensuring they are orientated in the same direction.

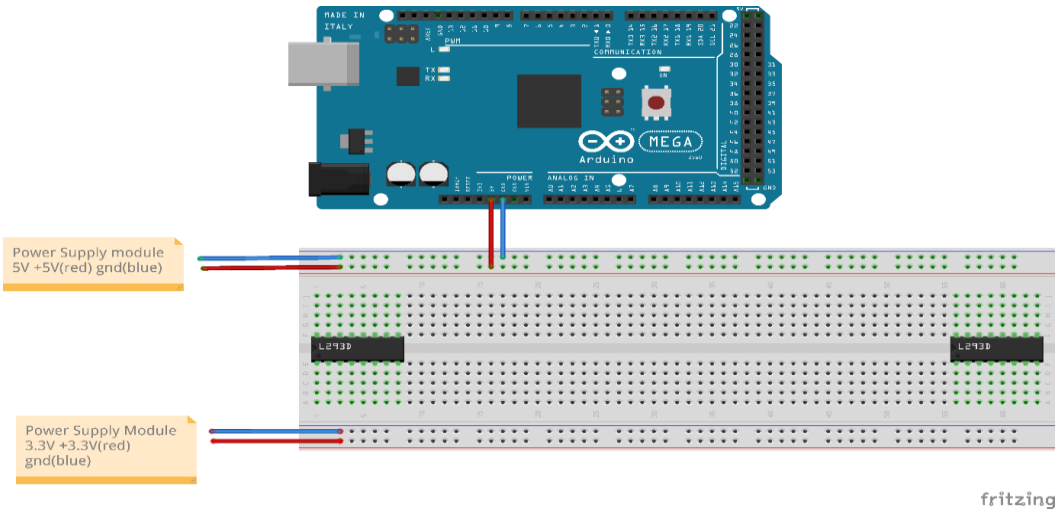


Figure 17

Connect the 4d systems display, 5V to 5V pin, gnd to gnd pin, pin 8 to reset pin and finally the Serial 1 communication pins to rx and tx (tx -> rx and rx-> tx). Connect the power supply once again and the screen should light up.

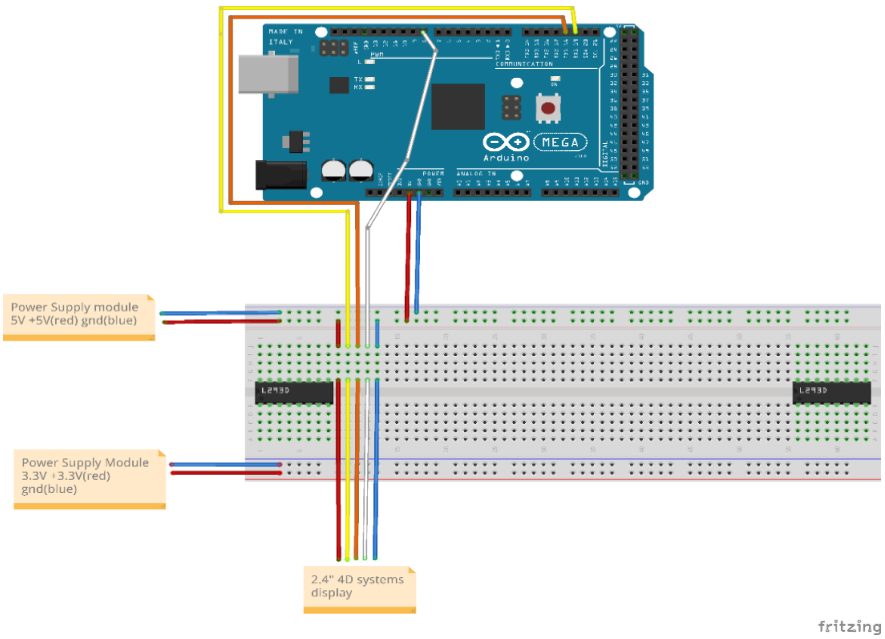
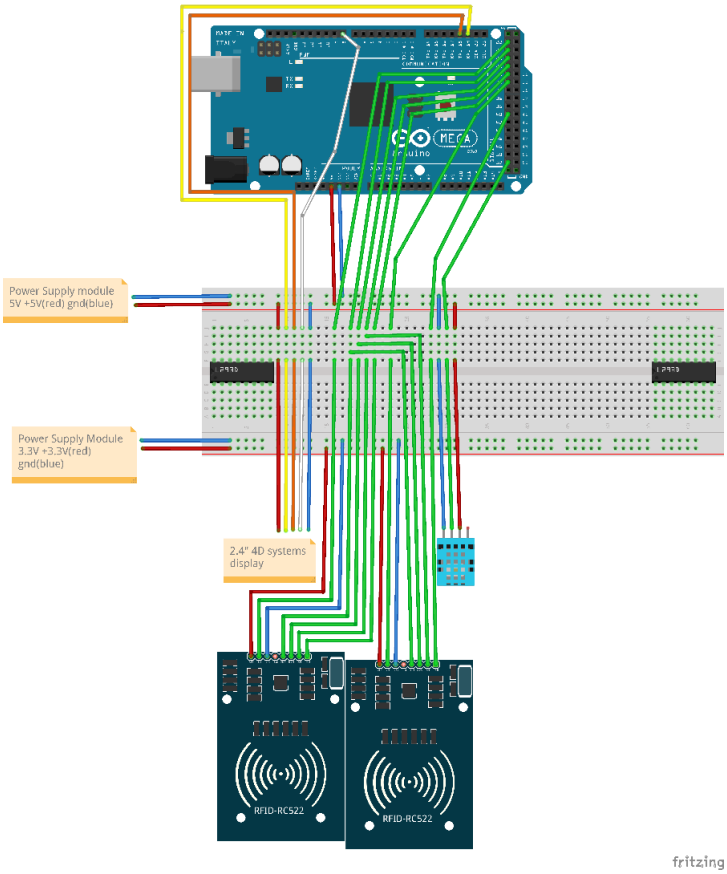


Figure 18

Next connect the 2 MFRC-522 sensors, note that these must be powered by the 3.3V power supply, and therefore must be connected to this and the corresponding ground wire. Each sensor requires 5 digital pins, Connect the SDA pin for the top RFID to pin 40 and RST for top RFID to pin 32, repeat for the bottom RDID, connecting to pins 30 and 22 respectively. Then, connect all the 3 remaining pins to pins 34, 36 and 38 for the top RFID and then pins 24, 26 and 28 for the bottom RFID. Connect the power supply and navigate to the 2nd page on the display. Place one of the male parts on each of the MFRC-522 sensors and if connected correctly, a message will appear on the screen validating the components.



Next, connect the DHT11 temperature and humidity sensor, this requires the left pin to be connected to the 5V power supply, centre pin to 52 digital pin and right pin to the ground. As illustrated in the diagram. Reconnect the power supply and initiate the system. Leave the first page activated and the temperature and humidity recorded should be displayed at the bottom left of the page.

Figure 19

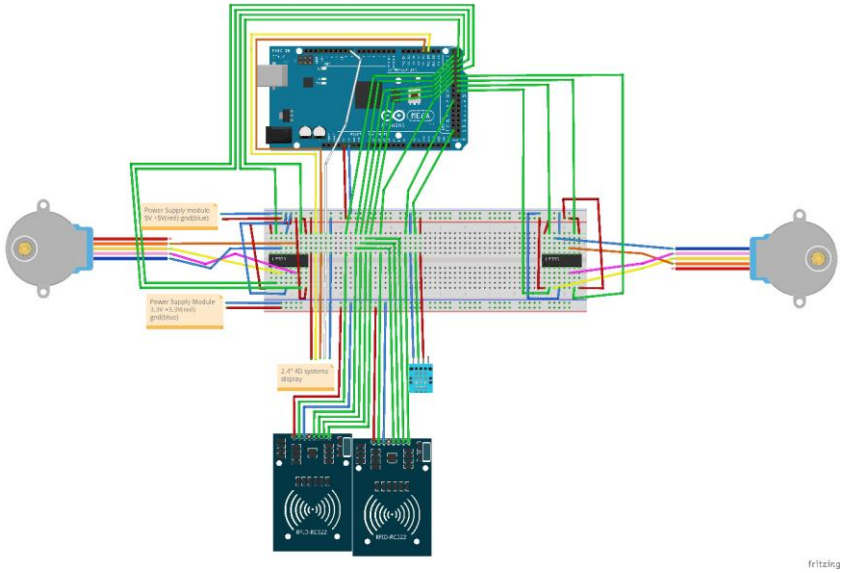


Figure 20

Next, connect the 2 stepper motors, each requires a L293D transistor. Begin by connecting the 5V power supply to the first and 8th pin on both sides on each transistor, and ground to pin 4 on each side. Connect digital pin 31 and 33 (23 and 29 for other motor) to either side on the 9th pin of the transistor. Then digital pins 37 and 35 (25 and 27) to either side of pin 2 of the transistor. Regarding the stepper motor wires, blue should be connected to 3rd from left top pin, orange to 6th from left top pin, yellow to 3rd from left bottom pin and pink to 6th from left top pin. Run a test on the machine, as if it were fully functional, but make sure to cancel before any damage is dealt to the machine.

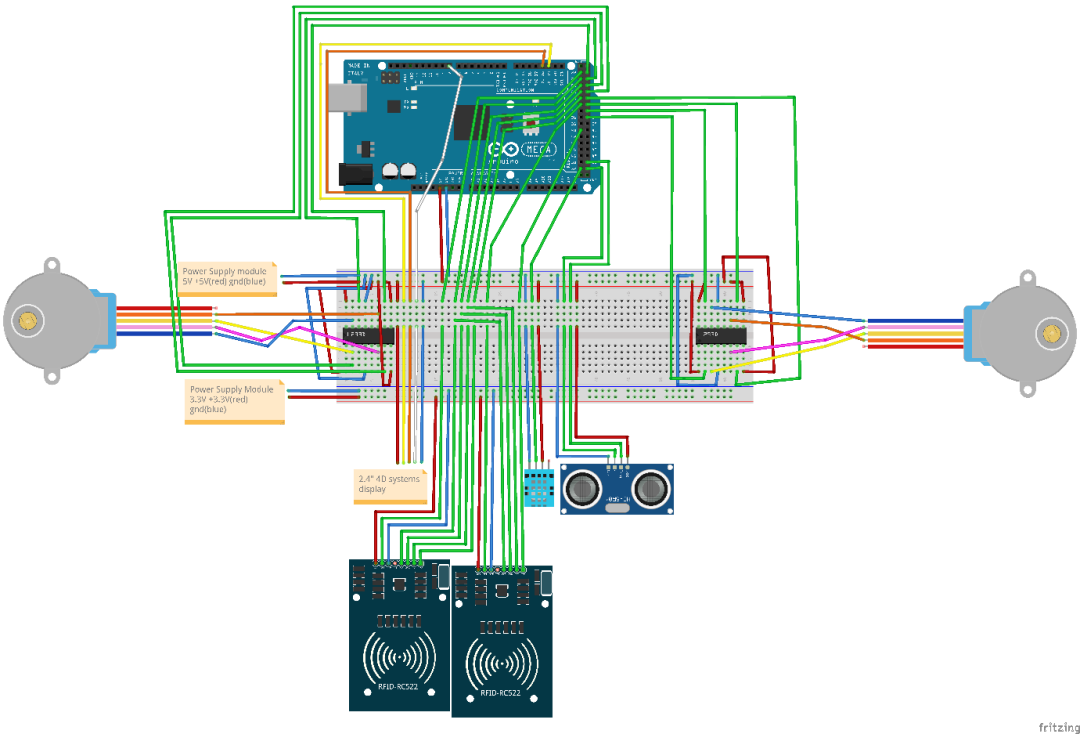


Figure 21

The HC-SR04 sensor also runs off 5V, connect the left pin to the 5V source and right pin to ground. Left centre should be connected to pin 51 digital and right centre to 53 digital. Run a tensile test as normal and observe that the ultimate tensile strain will be measured and shown in the results page.

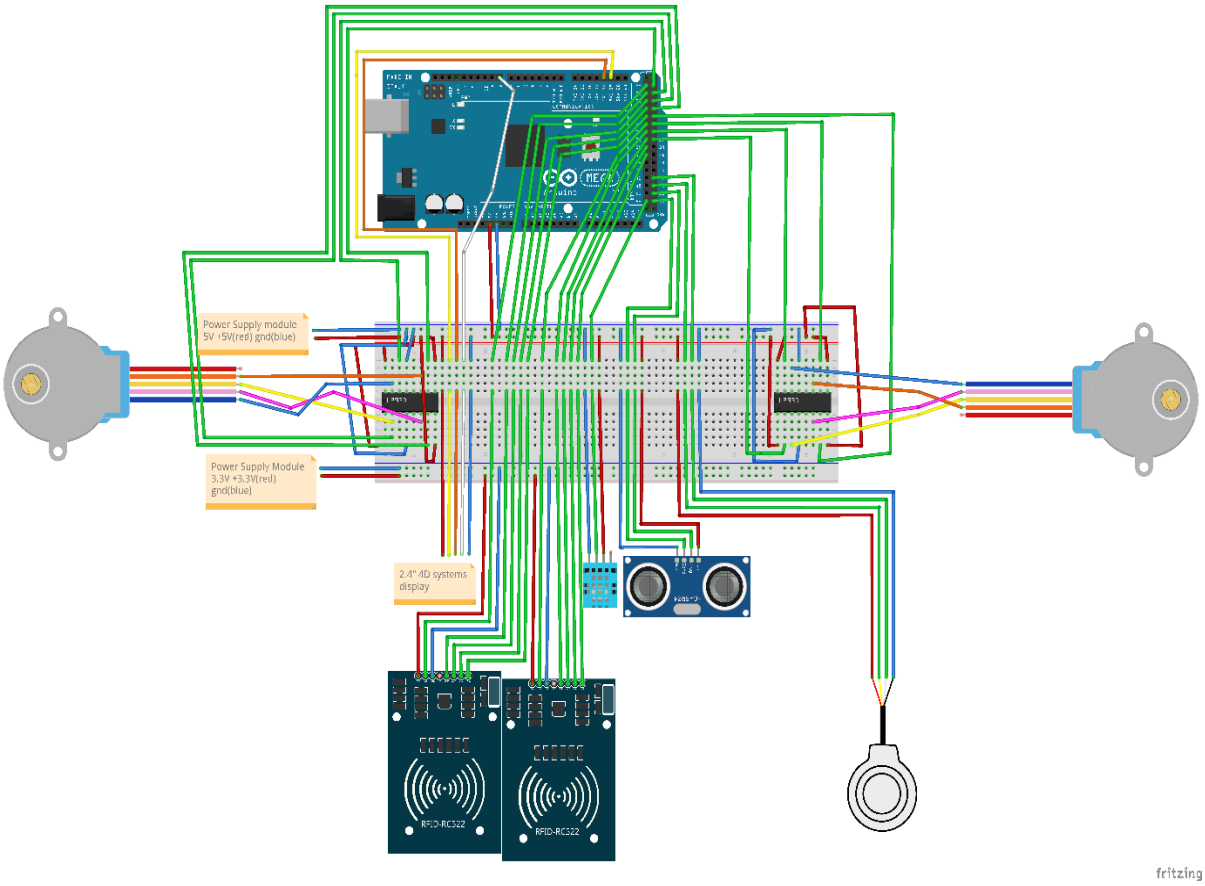


Figure 22

Finally, attach the load cell, the orange wire is to be connected to the 5V source, black to the ground, left centre to digital pin 47 and right centre to digital pin 49. Test the wiring by running a tensile test with a sample, and the line on the stress – time graph should change during the test.

User Interface

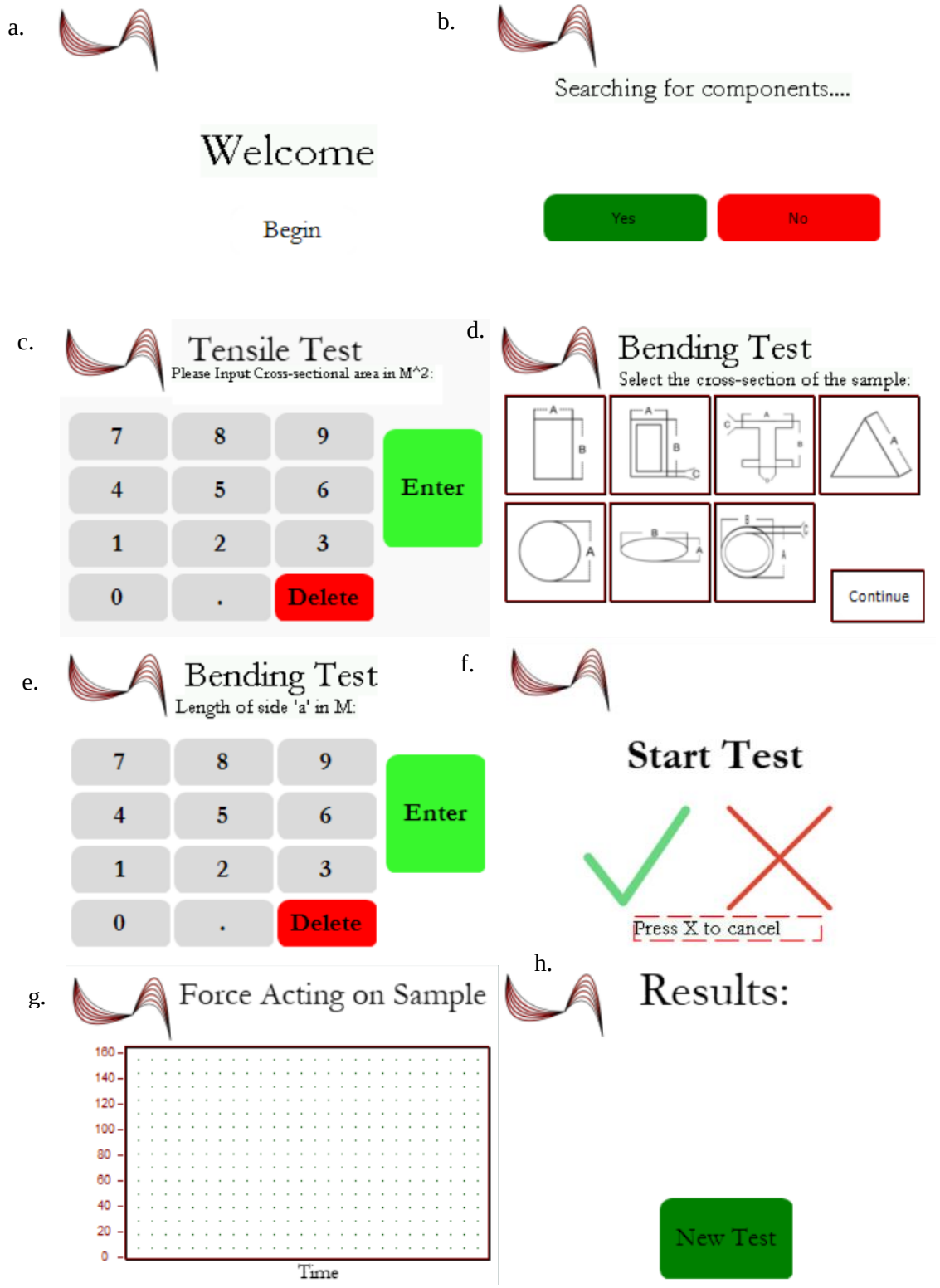


Figure 23: The User interface through which interaction with the machine will occur. Each image is a different form the screen will show. Page b. will allow for the male parts to be attached and detected. Then the next form will depend on the type of test that is being carried out, tensile or bending. G. is a stress time graph that will be plotted while the test is being carried out.

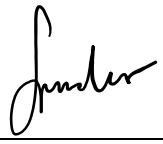
Equality, Diversity and Inclusion Report

Inclusivity was one of the key points the company wanted to excel on and to guarantee that everyone felt that they could speak up and be listened to when they needed help or had a problem. To ensure that those too shy to do so, we sent out a weekly questionnaire. In doing so, this also allowed us to highlight key problems within the company.

Communication was the greatest weaknesses in the company, and it had created many problems. In the design process the communication between subgroups was poor to the extent that members of the company didn't know who the subgroup leaders were or how the machine parts worked together or even what the parts were. To tackle this, we created a PowerPoint to explain what each subgroup was working on presented by the subgroup leader and we did this every week, ensuring that all members knew what was going on. In addition, we created a subgroup leader group chat to increase communication for report writing and deadlines which needed to be met. When manufacturing started, due to everyone's increased communication it ran a lot smoother, and many problems were solved by members of other subgroups.

Another issue which came up throughout the whole process was learning people's strength and using them best. The Director kept on changing the design of the machine, visually and functionally which caused great distress for many members in the company, after a discussion and the recognition of how difficult it was making it, these changed to conversations rather than completely redoing CAD models, which could be discussed before taken further. Also, some members of the team found it difficult to complete all the tasks designated to their role title and others felt they were completing tasks they weren't required to. This was discussed with the member and Director team multiple times, then after time when the member was still not keeping up with the demands, there was a reallocation of titles and movement within the company. This allowed members who found they had too much to do could focus on subgroup specific design rather than whole company tasks like the parts sheet. As a company we thought that this was also fairer as people got recognised for their work.

Overall, the company worked well to adapting how people worked and were open to new ways of tackling problems. As the contact hours with each other increased, especially in the second term, people got more comfortable with each other increasing productivity and making the whole team work more cohesively.

Amelia Spindler (she.her) Equality, Diversity and Inclusion Officer


Sustainability Report

This report will provide a final analysis of the carbon footprint and impact of design on the environment and outline improvements made to the previous design to render the product more sustainable on the market.

The initial design highlighted PMMA as the primary choice of material for the frame and mechanism. Although it is a synthetic polymer with relatively low toxicity to the environment during manufacturing, it was determined that the amount used could be reduced to reduce concerns associated with the end-of-life management. One of the replacements chosen for the use of PMMA in the adapter was wood. Wood is an incredibly useful renewable source, and as highlighted by Econation, 'the production and processing of wood is more energy efficient than most alternatives,' [2] meaning it has a very low carbon footprint. Additionally, Greencast 100% recycled acrylic sheets are used for the mechanism instead of normal PMMA sheets. This specific product is also 100% recyclable which is incredibly useful to reduce the environmental impact of the product as normal PMMA is very difficult to recycle. Therefore, the replacement of a significant amount of PMMA with wood and recycled acrylic reduces the overall carbon footprint of the design, and significantly reduces concerns associated with recycling of the product.

The product will make use of an RF cards and integrated circuits which compose of mainly Polypropylene (PP) and small amounts of PET. This is a replacement for the PET-based NFC stickers that were previously suggested. The amount of PET used in the design has decreased as there were concerns associated with the toxicity and lifespan of PET in landfills. It was determined that the production of polypropylene produces a lower carbon footprint than PET by approximately 2.5 –4.05 kg less of Co2 produced per kg of material [3] which is a significant reduction. Polypropylene also takes a ‘much shorter time to degrade than other types of plastic,’ [4] as well as not releasing any toxic chemicals during the recycling process. The combination of these two plastics as opposed to the sole use of PET was therefore determined to be a more sustainable option for the production and end of life management of the product.

Furthermore, as referenced in the previous report, although relatively small amounts of metal were used in the initial design, it would be useful to opt for metals with a lower carbon footprint. To accommodate for this the lead screw and thrust ball bearing are made from steel. Steel is often referred to as the ‘most recycled material in the world’,[3] as it can be reused infinitely, and it is a cradle-to-cradle material. As a result of these characteristics, it produces less waste than other metals and requires less energy usage to produce. Therefore, less CO2 is required to produce steel than other metals, and it is the most sustainable metal available for use in the design.

Changes made to the design to transition from a square shaped base to a circular base enabled the company to reduce the amount of material required to form the structure. Therefore, this also works to reduce the energy required to produce the product, and the carbon footprint of the design.

Overall, the changes made to the design and the materials chosen for the design improve the sustainability of the product and ensure that the end-of-life management has as little environmental impact as possible.

Safeeyah Ahmed (she.her) Chief of Sustainability
SAhmed

Financial Report

After the submission of the concept design report, a lot of modifications and changes have been made to the overall design of our machine. This also meant that we had to make some changes as well with some of our purchases moving forward. One of our substantial purchases was the leadscrew. Initially, the old rectangular box design required us to CNC cut it into four equal pieces for each corner to balance. Because of this, we found that we would need to purchase double the number of leadscrews and still have a lot of it left over, which would be a waste of material. Instead, we had decided to change our design so that the overall shape would be roughly triangular. As a result, we only had to purchase one leadscrew and cut it into three equal pieces and not worry about wasting money or any excess material.

Another large expenditure we made was the touch screen which allowed us to control the machine. Our previous plans to buy a touch screen from Robot shop got cancelled due to the unreliability of the supplier. The new screen that we found was from RS Components, but it was more costly when compared to the previous screen. However, since we saved a substantial amount of money with the leadscrew, we were able to fit the new touch screen in our budget.

Following the concept financial report, we did our research and found suitable PMMA (white and clear ones since they were the cheapest) and wood sheets for laser cutting. The adapters subgroup opted to use wood to make the adapter parts because wood makes it look aesthetically pleasing and sustainable for the environment. The rest of the machine was 3D printed using the PLA that the department provided and laser cut using the PMMA sheets bought. These materials are all considered a “win-win” on financial and sustainability aspect.

We also built relationships between companies by creating a collaborating ordering spreadsheet which was open to other companies weekly, minimising the delivery fees associated with ordering parts. Processing costs such as cutting the leadscrews, would have cost us £10 per hour, but it was prevented by using interdepartmental service from the STW workshop.

In conclusion, as seen from the Company 7 Budget spreadsheet, most of our money was used towards the coding subgroup, since essential parts were required to meet the criteria of the machine. Because of this, we had to overcome the challenge of managing our remaining budget on the larger expenditures such as the leadscrew for the movement mechanism subgroup and the laser cutting sheets for the main frame and adapters subgroup. Our overall spending came close to the given budget of 150 pounds even after completely changing our initial design. However, despite these challenges, we have managed to spend our budget efficiently with still a few pounds leftover.

Su Peng Wang (he.him) Chief Financial Officer
Su Peng Wang

GANTT Chart

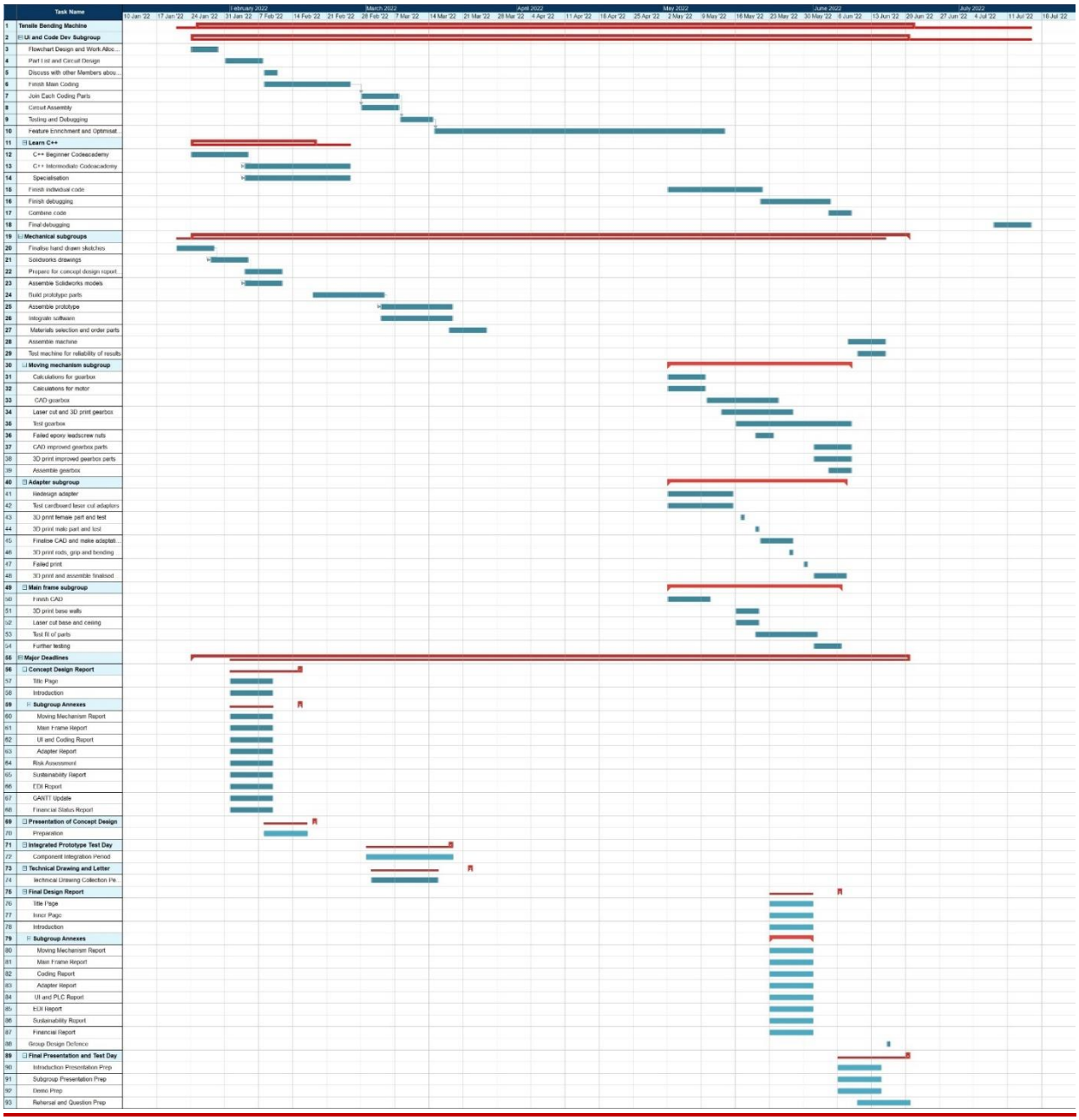


Figure 24: An updated GANTT chart that matches the company's progress through the testing and manufacturing phase. The two phases occurred almost simultaneously as we followed a cycle of fabricating parts, testing them, recalibrating our dimensions, and repeating.

Assessment of Status of Project

Although Bensile is consisted of many talented individuals, there were some hurdles that were unforeseen that needed to be solved, such as 3D printing errors when printing the walls due to z axis misplacement, which was solved by exploiting the mismatch in heights by using it as a slit for temperature and humidity sensors. Or the lack of designated holes for wiring that go across the machine from top to bottom was solved by channelling them down along the aluminium bars. Furthermore, there are some problems that we are still facing such as, some parts still require fabrication and ultimately, assembling the whole machine and testing it out to take account of physical factors such as friction, which may deviate from theoretical calculation.

However, there are also great feats that we have made during this period. Firstly, despite not having prior knowledge about coding in C, we now have working code for our TFT display. This shows a great amount of independent learning and determination from our coding subgroup. Secondly, our team demonstrated resourcefulness when faced with unexpected challenges. For instance, when we realised that we needed more torque for the bending and tensile tests, we immediately went to work on a new gearbox system. Even though a planetary gear system seemed intimidating at first, the moving mechanism group spent time to learn about the mechanics behind it and designed a system of their own. Thirdly, our team was strategic in the way we approached the fabrication of parts. Knowing that we had a limited amount of resources led to us testing the fit of our parts using small portions of it before printing or cutting the entire part. This played a huge role in allowing us to stay below our budget. Lastly, our team showed innovation in designing. We created a loadcell box that allowed the loadcell to measure loads in a bilateral direction and integrated it with the moving platform, which was a great engineering feat. Moreover, the clover design of our main frame is unique and functional.

Bensile is a company that strives to continually improve, so we have adopted a framework shown in Figure ??? throughout the testing and final phase of this project. As we near the end of this project, we are confident to be able to deliver a product that meets the needs of our clients.

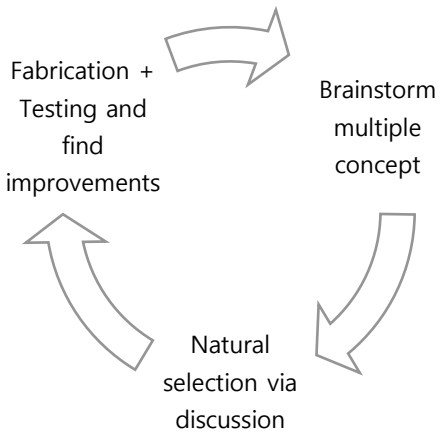


Figure 25: Cyclical framework used in the testing and manufacturing phases to ensure continual improvements to be made.

Minjae Kim (he.him) Director	Natalie Ngoh (they.them) Deputy Director

Coding Appendix

Appendix 1

```
#include <genieArduino.h>
#include <SPI.h>
#include <MFRC522.h>
//need to define pins for display

//define pins for MFRC522 reader(add numbers for both readers)
#define SS_1_PIN 9
#define SS_2_PIN 10
#define RST_PIN 6
MFRC522 rfid(SS_1_PIN, RST_PIN); // Instance of the class
MFRC522 rfid1(SS_2_PIN, RST_PIN); // Instance of the class



//-----------------------------------------------------------------------------------------------------------------------
//set of global variables to be used in the code
// put your setup code here, to run once:
int Form = 0;
String Test;
String Test1;
//Need all the hex strings for the nfc tags
String Tensile1;
String Tensile2;
String Bending;
String point3;
String point4;
String Value;
bool Output = true ;
float valstore;
String Shape;
float TArea;
String str;
float a;
float b;
float c;
String measr = "a";
float flt5;
String str5 = "";
int i = 0;
int decimal = 0;
int formstr;


Genie genie;
#define RESETLINE 8
void setup() {
  Serial.begin(9600);
  Serial1.begin(9600);
  Serial.println("Serial activates");
  genie.Begin(Serial1);
  genie.AttachEventHandler(myGenieEventHandler);
  pinMode(8, OUTPUT);
  digitalWrite(8, 0);//Reset the display via D2
  delay(100);
  digitalWrite(8, 1);
  delay(5000); //let the display start up after reset(large delay time as large project)(display will not respond so have increased start up time
  genie.WriteContrast(1); //Set contrast of display, may be basic(0 or 1) due to nature of display
  Serial.println("Finish setup");
  //setup the MFRC522 module, check if i should do multiple serial begins for each component
//  SPI.begin;//init SPI bus
//  rfid.PCD_Init();// init MFRC522
//  rfid.PCD_Init();//init other MFRC522




}


//when the 'form the display is in' is checked, I have stored the form stored in a variable, and just change this value whenever I change the form in the code.
//all the buttons will report a message when pressed.


//-----------------------------------------------------------------------------------------------------------------------
```

```
  Serial.println("next Form 5");

  //Form 5 Wait for confirmation of start test start selected test and display form 6
  while(Form == 5){
    genie.DoEvents();
    delay(100);
  }
  Serial.println("next Form 6");
  //Form 6 Display form until test is completed
  while(Form == 6){
    delay(5000);
    genie.WriteObject(GENIE_OBJ_FORM, 7, 0);   //unfinished code
    Form = 7;
  }
  // Form 7 Using the calculated values, display these and loop to wait for the option to start new test. This takes you to Form 1 and erases all previously stored data
  Serial.println("next Form 7");
  while(Form == 7){
    genie.DoEvents();
    delay(100);
  }
  Serial.println("next Form 8");
  // Form 8, function to export data should be here.
  while(Form == 8){
    genie.DoEvents();
    delay(100);
  }



}
//--------------------------------


//Event handler command
void myGenieEventHandler(void){             //eventhandler function has multiple uses. It goes through and chechs what form the system is in, then the button that has been pressed. Depending on the button, it will either change form, update a
                                            //The eventhandler and the code here works, but for some reason when the genie.WriteObject is called, nothing happens
  Serial.println("Event being processed");  //all the serial prints were to test whether the code functioned or not
  Serial.println(Form);
  genieFrame Event;
  genie.DequeueEvent(&Event);          //Remove next queued event from the buffer to process it below

    //only one event occuring in form 0
  if(genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_WINBUTTON, 32)){
    Serial.println("Winbutton 32 detected");
    genie.WriteObject(GENIE_OBJ_FORM, 1, 0);//change to form 1
    delay(200);
    Form = 1;
    return;


    }

  if(Form == 1){            //determines which form the screen is in
    if(genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_WINBUTTON, 0)){
      Serial.println("winbutton 0 detected");
      genie.WriteObject(GENIE_OBJ_FORM, 3, 0); //change to form 2
      delay(200);
      Form = 3;
      return;
    }

    if(genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_WINBUTTON, 1)){
      str = "Please place new components";            //user has pressed cross therefore new components are to be placed
      genie.WriteStr(0, str); //message prompt on screen
      delay(200);
      return;
    }
    return;
  }

  if(Form == 2 or Form == 4){                     //this is essentially a keypad, as the built in one did not seem to be working so I have made my own rudimental one.
    Serial.println(Shape);
    Serial.println("Form 2 or 4 detected");
    if(genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_WINBUTTON, 31) or genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_WINBUTTON, 3)){ //The enter button, storage of the number entered.
      if(Form == 2){
        TArea = flt5;
        str = "Value stored";
        genie.WriteStr(2, str);       //change string displayed
        delay(1000);
        genie.WriteObject(GENIE_OBJ_FORM, 5, 0);   //change to form 5
        delay(200);
        Serial.println("Next form for both 2 and 4");
        Form = 5;
        decimal = 0;
        return;
      }
      if(Form == 4){
        if(Shape == "Circle" or Shape == "Triangle"){   //checks the shape of the bending sample
          str = "Length of side 'a' in M: ";    //Change the string to display the number inputed for the length of a side
          str.concat(str5);
          genie.WriteStr(4, str);
          a = flt5;                             //there are multiple measurements to each shape
          str = "Length of side a stored";
          genie.WriteStr(4, str);
          delay(1000);
          genie.WriteObject(GENIE_OBJ_FORM, 5, 0);
          delay(200);
          str = "Length of side 'a' in M: ";
          genie.WriteStr(4, str);
          Form = 5;

          decimal = 0;
          str5 = "0";
          return;
        }
        if(Shape == "Rectangle" or Shape == "Elipse"){
          if(measr == "b"){   //value of side b is stored for rectangle or elipse
            str = "Length of side 'b' in M: ";
            str.concat(str5);
            genie.WriteStr(4, str);
            delay(500);
            measr = "a";
            b = flt5;
            str = "Value of b stored";
            genie.WriteStr(4, str);
            delay(1000);
            genie.WriteObject(GENIE_OBJ_FORM, 5, 0);   //change to form 5
            delay(200);
            str = "Length of side 'a' in M: ";
            genie.WriteStr(4, str);
            Form = 5;
            decimal = 0;
            str5 = "0";
            return;
          }
          if(measr == "a"){  //value for side a stored for rectangle or elipse
            str = "Length of side 'a' in M: ";
            str.concat(str5);
            genie.WriteStr(4, str);
            delay(500);
            a = flt5;
            measr = "b";
            str = "Value for 'a' stored";
            genie.WriteStr(4, str);
            delay(1000);
            str = "Length of side 'b' in M: "; //input of next length
            genie.WriteStr(4, str);
            decimal = 0;
            str5 = "0";
            return;
          }
        }
        if(Shape == "Beam" or Shape == "HRectangle" or Shape == "HCircle"){
          if(measr == "c"){   //value for side c stored for beam, hollow rectangle and hollow circle
            str = "Length of side 'c' in M: ";
            str.concat(str5);
            genie.WriteStr(4, str);
            c = flt5;
            measr = "a";
            str = "Value of c stored";  //input of next length
            genie.WriteStr(4, str);
```

```
        delay(1000);
        genie.WriteObject(GENIE_OBJ_FORM, 5, 0);//Change form 5
        delay(200);
        str = "Length of side 'a' in M: ";
        genie.WriteStr(4, str);
        Form = 5;
        decimal = 0;
        str5 = "0";
        return;
        }
      if(measr == "b"){          //value for side b stored for beam, hollow rectangle and hollow circle
        str = "Length of side 'b' in M: ";
        str.concat(str5);
        genie.WriteStr(4, str);
        b = flt5;
        flt5 = 0;
        measr = "c";
        str = "Value of b stored";  //inpur of next length
        genie.WriteStr(4, str);
        delay(1000);
        str = "Length of side 'c' in M: ";
        genie.WriteStr(4, str);
        delay(200);
        decimal = 0;
        str5 = "";
        return;
      }
      if(measr == "a"){    //value for side a stored for beam, hollow rectangle and hollow circle
        str = "Length of side 'a' in M: ";
        str.concat(str5);
        genie.WriteStr(4, str);
        a = flt5;
        flt5 = 0;
        measr = "b";
        str = "Value for a stored";
        genie.WriteStr(4, str);
        delay(1000);
        str = "Length of side 'b' in M: "; //allows for input of next length
        genie.WriteStr(4, str);
        delay(200);
        decimal = 0;
        str5 = "0";
        return;
      }
    }
  }
}
if(genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_WINBUTTON, 30) or genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_WINBUTTON, 2)){ //delete button
  Serial.println("delete");
  str5.remove(str5.length()-1);   //removes the last value inputed
  if(Form == 2){
    str = "Cross-sectional area in M^2: ";
    str.concat(str5);
    genie.WriteStr(2, str);   //update the value displayed on the screen
    Serial.println(flt5);
    Serial.println(str);
    delay(200);
    return;
  }
  if(Form == 4){
    str = "Length of side '" + measr + "' in M: ";
    str.concat(str5);
    Serial.println(flt5);
    Serial.println(str);
    genie.WriteStr(4, str);   //update the value displayed on the screen
    delay(200);
    return;
  }
  return;
}

 //all the next winbutton if statements will take a value, update the string and then update the string displayed on the screen. it will check the form the display is in and change the corresponding string in that form
 //the valverif function is finished, I just have in another code document. Essentially it juct verifies a valid number is inputed e.g. not 0.5.3333


if(genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_WINBUTTON, 17) or genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_WINBUTTON, 29)){ //.
  Value = ".";
  Serial.println(Value);
  Serial.println(decimal);
  if(decimal == 0){
    str5.concat(Value);
    decimal ++;
  }
  if(decimal != 0){
    str = "Invalid input, try again";
    genie.WriteStr(2, str);
  }
  if(Form == 2){                          //check form
    str = "Cross-sectional area in M^2: ";
    str.concat(str5);
    genie.WriteStr(2, str);   //update string
    delay(200);
    return;
    }
  if(Form == 4){
    str = "Length of side '" + measr + "' in M: ";
    str.concat(str5);
    genie.WriteStr(4, str);          //update string
    delay(200);
    return;
  }
  return;
}
if(genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_WINBUTTON,18) or genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_WINBUTTON, 28)){ //0
  Value = "0";
  Serial.println(Value);
  str5.concat(Value);
  if(Form == 2){                          //check form
    str = "Cross-sectional area in M^2: ";
    str.concat(str5);
    genie.WriteStr(2, str);           //update string
    delay(200);
    return;
    }
  if(Form == 4){
    str = "Length of side '" + measr + "' in M: ";
    str.concat(str5);
    genie.WriteStr(4, str);           //update string
    delay(200);
    return;
  }
  return;
}
if(genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_WINBUTTON, 16) or genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_WINBUTTON, 25)){ //1
  Value = "1";
  Serial.println(Value);
  str5.concat(Value);
  if(Form == 2){                          //check form
    str = "Cross-sectional area in M^2: ";
    str.concat(str5);
    genie.WriteStr(2, str);           //update string
    delay(200);
    return;
  }
  if(Form == 4){
    str = "Length of side '" + measr + "' in M: ";
    str.concat(str5);
    genie.WriteStr(4, str);           //update string
    delay(200);
    return;
  }
  return;
}
if(genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_WINBUTTON, 15) or genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_WINBUTTON, 26)){ //2
  Value = "2";
  Serial.println(Value);
  str5.concat(Value);
  if(Form == 2){                          //check form
    str = "Cross-sectional area in M^2: ";
    str.concat(str5);
```

```
    genie.WriteStr(2, str);          //update string
    delay(200);
    return;
  }
  if(Form == 4){
    str = "Length of side '" + measr + "' in M: ";
    str.concat(str5);
    genie.WriteStr(4, str);       //update string
    delay(200);
    return;
  }
  return;
}
if(genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_WINBUTTON, 14) or genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_WINBUTTON, 27)){ //3
  Value = "3";
  Serial.println(Value);
  str5.concat(Value);
  if(Form == 2){                          //check form
    str = "Cross-sectional area in M^2: ";
    str.concat(str5);
    genie.WriteStr(2, str);         //update string
    delay(200);
    return;
  }
  if(Form == 4){
    str = "Length of side '" + measr + "' in M: ";
    str.concat(str5);
    genie.WriteStr(4, str);           //update string
    delay(200);
    return;
  }
  return;
}
if(genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_WINBUTTON, 13) or genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_WINBUTTON, 22)){ //4
  Value = "4";
  Serial.println(Value);
  str5.concat(Value);
  if(Form == 2){                                //check form
    str = "Cross-sectional area in M^2: ";
    str.concat(str5);
    genie.WriteStr(2, str);        //update string
    delay(200);
    return;
  }
  if(Form == 4){
    str = "Length of side '" + measr + "' in M: ";
    str.concat(str5);
    genie.WriteStr(4, str);          //update string
    delay(200);
    return;

  }
  return;
}
if(genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_WINBUTTON, 12) or genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_WINBUTTON, 23)){ //5
  Value = "5";
  Serial.println(Value);
  str5.concat(Value);
  if(Form == 2){                          //check form
    str = "Cross-sectional area in M^2: ";
    str.concat(str5);
    genie.WriteStr(2, str);            //update string
    delay(200);
    return;
  }
  if(Form == 4){
    str = "Length of side '" + measr + "' in M: ";
    str.concat(str5);
    genie.WriteStr(4, str);          //update string
    delay(200);
    return;
  }
  return;
}
if(genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_WINBUTTON, 11) or genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_WINBUTTON, 24)){ //6
  Value = "6";
  Serial.println(Value);
  str5.concat(Value);
  if(Form == 2){                          //check form
    str = "Cross-sectional area in M^2: ";
    str.concat(str5);
    genie.WriteStr(2, str);         //update string
    delay(200);
    return;
  }
  if(Form == 4){
    str = "Length of side '" + measr + "' in M: ";
    str.concat(str5);
    genie.WriteStr(4, str);        //update string
    delay(200);
    return;
  }
  return;
}
if(genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_WINBUTTON, 6) or genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_WINBUTTON, 19)){ //7
  Value = "7";
  Serial.println(Value);
  str5.concat(Value);
  if(Form == 2){                                //check form
    str = "Cross-sectional area in M^2: ";
    str.concat(str5);
    genie.WriteStr(2, str);          //update string
    delay(200);
    return;
  }
  if(Form == 4){
    str = "Length of side '" + measr + "' in M: ";
    str.concat(str5);
    genie.WriteStr(4, str);            //update string
    delay(200);
    return;
  }
  return;
}
if(genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_WINBUTTON, 5) or genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_WINBUTTON, 20)){ //8
  Value = "8";
  Serial.println(Value);
  str5.concat(Value);
  if(Form == 2){                                //check form
    str = "Cross-sectional area in M^2: ";
    str.concat(str5);
    genie.WriteStr(2, str);           //update string
    delay(200);
    return;
  }
  if(Form == 4){
    str = "Length of side '" + measr + "' in M: ";
    str.concat(str5);
    genie.WriteStr(4, str);           //update string
    delay(200);
    return;
  }
  return;
}
if(genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_WINBUTTON, 10) or genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_WINBUTTON, 21)){ //9
  Value = "9";
  Serial.println(Value);
  str5.concat(Value);
  if(Form == 2){                                //check form
    str = "Cross-sectional area in M^2: ";
    str.concat(str5);
    genie.WriteStr(2, str);             //update string
    delay(200);
    return;
  }
  if(Form == 4){
    str = "Length of side '" + measr + "' in M: ";
    str.concat(str5);
    genie.WriteStr(4, str);           //update string
    delay(200);
    return;
```

```
      }
      return;
    }
    return;
  }
  if(Form == 3){                                        //Takes input of the cross sectional shape of the sample
    //function that stores input of shape
    if(genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_USERBUTTON, 25)){
      Shape = "Rectangle";
      str = "Rectangle Selected";
      genie.WriteStr(5,str);
      delay(200);
      return;
    }
    if(genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_USERBUTTON, 26)){
      Shape = "Triangle";
      str = "Triangle Selected";
      genie.WriteStr(5,str);
      delay(200);
      return;
    }
    if(genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_USERBUTTON, 27)){
      Shape = "Circle";
      str = "Circle Selected";
      genie.WriteStr(5,str);
      delay(200);
      return;
    }
    if(genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_USERBUTTON, 28)){
      Shape = "Elipse";
      str = "Elipse Selected";
      genie.WriteStr(5,str);
      delay(200);
      return;
    }
    if(genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_USERBUTTON, 29)){
      Shape = "HCircle";
      str = "Hollow Circle Selected";
      genie.WriteStr(5,str);
      delay(200);
      return;
    }
    if(genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_USERBUTTON, 30)){
      Shape = "HRectangle";
      str = "Hollow Rectangle Selected";
      genie.WriteStr(5,str);
      delay(200);
      return;
    }
    if(genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_USERBUTTON, 31)){

      Shape = "Beam";
      str = "Beam Selected";
      genie.WriteStr(5,str);
      delay(200);
      return;
    }
    if(genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_WINBUTTON, 4)){
      Form = 4;
      genie.WriteObject(GENIE_OBJ_FORM, 4, 0);//Change to form 4
      delay(200);
      return;
    }
    return;
  }
  if(Form == 5){   //Start the test or cancel and reset the machine
    if(genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_USERBUTTON, 1)){
      Serial.println("userbutton 1 detected");
      genie.WriteObject(GENIE_OBJ_FORM, 6, 0);  //change to form 6
      str = "Select the cross-section of the sample:";
      genie.WriteStr(5, str);
      delay(200);
      Form = 6;
      return;
    }
    if(genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_USERBUTTON, 2)){
      genie.WriteObject(GENIE_OBJ_FORM, 0, 0);   //cancel the procedure, change to form 0
      delay(200);
      Form = 0;
      measr = "a";
      flt5 = 0;
      str5 = "";
      return;
    }
    return;
  }
  if(Form == 7){                                  //Reset the machine to start a new test or option to export data to a computer
    if(genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_WINBUTTON, 7)){
      Form = 8;
      genie.WriteObject(GENIE_OBJ_FORM, 8, 0);  //change to form 8
      delay(200);
      return;
    }
    if(genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_WINBUTTON, 8)){
      Form = 0;
      measr = "a";
      str5 = "";
      genie.WriteObject(GENIE_OBJ_FORM, 0, 0); //change to form 0
      delay(200);
      return;
    }
    return;
  }
  if(Form == 8){
      //Export funciton should be here
    if(genie.EventIs(&Event, GENIE_REPORT_EVENT, GENIE_OBJ_WINBUTTON, 9)){
      Form = 0;
      measr = "a";
      str5 = "";
      genie.WriteObject(GENIE_OBJ_FORM, 0, 0); //change to form 0
      delay(200);
      return;
    }
    return;
  }
  return;
}
```

Appendix 2

```
#include "SPI.h"
#include "MFRC522.h"


const int PIN_SDA = 10;

const int PIN_RST = 9;

MFRC522 nfc2(PIN_SDA, PIN_RST);

void setup() {
  Serial.begin(9600);
  SPI.begin();
  nfc2.PCD_Init();


}

void loop() {
  if (!nfc2.PICC_IsNewCardPresent())

  {

    return;

  }

    if(!nfc2.PICC_ReadCardSerial())

  {

    return;

  }

  String type2 = "";
      Serial.print("RFID TAG ID:");
      for (byte i = 0; i < nfc2.uid.size; ++i) {



   Serial.print(nfc2.uid.uidByte[i] < 0x10 ? " 0" : " ");
   Serial.print(nfc2.uid.uidByte[i], HEX);
   type2.concat(String(nfc2.uid.uidByte[i] < 0x10 ? " 0" : " "));
   type2.concat(String(nfc2.uid.uidByte[i], HEX));        //hex UID taken from the NFC tag are converted into a string

        }
  Serial.println();
  Serial.print("Message : ");
  type2.toUpperCase();
    if (type2.substring(1) == "13 87 0C 0C") //UID of detected NFC is compared to known UID of the male components
  {
    Serial.println("Tensile 1");
    Serial.println();
    delay(300);
  }
    else if (type2.substring(1) == "83 93 E7 0B")
  {
    Serial.println("Tensile 2");
    Serial.println();
    delay(300);
  }
    else if (type2.substring(1) == "B3 ED 11 0C")
  {
    Serial.println("Bending bottom");
    Serial.println();
    delay(300);
  }

    else if (type2.substring(1) == "53 74 F6 0B")
  {
    Serial.println("Bending top");
    Serial.println();
    delay(300);
  }
 else   {
    Serial.println("Attachemnt not recognised");
    delay(300);
  }
}
//Data analysis
void setup () {

Serial.begin(9600);

}

void loop (){
// example values

float stress [] = {0, 10, 20, 30, 40, 50, 60, 70, 80, 90, 100};

float strain []={0.0, 20.0, 40.0, 60.0, 80.0, 100.0, 120.0, 140.0, 160.0, 180.0, 200.0};

float x1 = strain[0];
float y1 = stress[0];
float x2 = strain[(sizeof(strain)/sizeof(float))-1];
float y2 = stress[(sizeof(stress)/sizeof(float))-1];

float gradient = (y2 - y1)/(x2-x1);

Serial.print(x1);
delay (1000);
Serial.println();
delay (1000);
Serial.print(x2);
delay (1000);
Serial.println();
delay (1000);
Serial.print(y1);
delay (1000);
Serial.println();
delay (1000);
Serial.print(y2);
delay (1000);
Serial.println();
delay (1000);
Serial.print(gradient);
delay (1000);
```

```
#include "DHT.h"
#define DHTPIN 2
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);

// Motor setup
#include <Wire.h>
#include <math.h>
#include <Adafruit_MotorShield.h>
#include "utility/Adafruit_MS_PWMServoDriver.h"
Adafruit_MotorShield AFMS = Adafruit_MotorShield();
Adafruit_DCMotor *myMotor = AFMS.getMotor(1); // number is the port

// Ultrasonic sensor setup
#define echoPin 2 // attach pin D2 Arduino to pin Echo of HC-SR04
#define trigPin 3 //attach pin D3 Arduino to pin Trig of HC-SR04
long duration; // variable for the duration of sound wave travel (in microseconds)
float distance; // variable for the distance measurement (in cm)

// Compression load cell setup
#include "FX29K.h"
FX29K load_cell(FX29K0, 0010, &Wire); // must set up

// NUMBER OF TIMES WE GO THROUGH THE LOOP
// Create variable for maximum number of motor steps
const int STEPS = 640; // will need to calibrate when machine is built, let run and see how long runs
// Count iterations of loop
int count = 0;

// INPUTS
// Create variable for axis identification axis_identification user input
int axis_identification; // 0 for true stress/strain and 1 for engineering stress/strain
// create variable for cylinder user input
int is_cylinder; // 1 for cylinder, 0 for cuboid
// create variable for sample width
float sample_width;
// Create variable for sample thickness sample_thickness
float sample_thickness;
// create variable for sample diameter
float sample_diameter;
// Create varaible for initial sample length initial_sample_length
float initial_sample_length;

// INITIAL CALCULATIONS
// create variable for radius
float radius;
// create variable for crossectional area
float crossec_area;
//create variable for crossectional area cylinder
float crossec_area_cyl;
// create variable for the current distance to the moving plate
float current_plate_position;
// create array for deflection (in cm) measured by ultrasonic sensor
float deflection[STEPS];
// Create array for temperature
float temperature[STEPS];
// Create array for humidity
float humidity[STEPS];
// create array for engineering strain
float eng_strain[STEPS];
// create array for true strain
float true_strain[STEPS];
// creare array for engineering stress
float eng_stress[STEPS];
// create array for true stress
float true_stress[STEPS];
// create array for enginering stress cylinder
float eng_stress_cyl[STEPS];
// create array for load
float load[STEPS];
// create array for force
float force[STEPS];
// create array for power
//float power[STEPS];

// CONSTANTS
// create constant pi
const float pi = 3.14159265359;
// create constant g
const float g = 9.81;
// create constant for height of machine
// const float machine_h = // in cm, find out value from design group;

void setup() {
    // put your setup code here, to run once:
  Serial.begin(9600);

  dht.begin(); // initialising temp and humidity sensor

  AFMS.begin(); // initialise motor object

  // setup ultrasonic sensor pin modes
  pinMode(trigPin, OUTPUT); // Sets the trigPin as an OUTPUT
  pinMode(echoPin, INPUT); // Sets the echoPin as an INPUT

  // tare compression load cell
  void load_cell.tare(); // will have to tare to a non zero value to account for the weight of the components

  // Get input for axis identification and store in variable axis_identification

  // Get input thickness of sample and store in variable sample_thickness
  // Get input sample length from LIDAR and store in variables initial_sample_length
void loop() {
  if (count == 0) { // first time through the loop
    // TODO: setup touchscreen UI
    // TODO: get inputs and store in variables
      // TODO: get user input axis_identification
      // TODO: get user input is_cylinder
      // TODO: get user input sample_width
      // TODO: get user input sample_thickness
      // TODO: get user input sample_diameter
      // TODO: get user input for initial_sample_length

    // Initial calculations based on inputs

    // calculate radius of sample from the diameter
    radius = sample_diameter / 2;

    // calculate crossectional area
    crossec_area = sample_width * sample_thickness;

    // depending on if sample is cylinder or not
    if (is_cylinder == 1){
      // calculate crossectional area cylinder
      crossec_area_cyl = pi * pow(radius, 2)
    }

  }

  // read humidity and store in humidity array [count]
  float humidity[count] = dht.readHumidity();
  // read temperature and store in temperature array [count]
  float temperature[count] = dht.readTemperature();

  // read deflection using the ultrasonic sensor
  // Clears the trigPin condition
  digitalWrite(trigPin, LOW);
  delayMicroseconds(2);
  // Sets the trigPin HIGH (ACTIVE) for 10 microseconds
  digitalWrite(trigPin, HIGH);
  delayMicroseconds(10);
  digitalWrite(trigPin, LOW);
  // Reads the echoPin, returns the sound wave travel time in microseconds
  duration = pulseIn(echoPin, HIGH);
  // Calculating the deflection
  deflection[count] = duration * 0.034 / 2; // Speed of sound wave divided by 2 (go and back)

  // set speed of motor from 0 to 255
  if (count == 0){
    myMotor->setSpeed(20);
    myMotor->run(FORWARD);
  }

  // Read load
  force[count] = float load_cell.getGrams();
```

```
  load[count] = (force[count] / 1000) * g;


  // calculate engineering stress
  if (is_cylinder == 1) {
    // calculate engineering stress cylinder
    eng_stress_cyl[count] = load[count] / crossec_area_cyl;
  } else {
    // calculate engineering stress
    eng_stress[count] = load[count] / crossec_area;
  }

    // calculate engineering strain
    eng_strain[count] = deflection[count] / initial_sample_length;

  if (axis_identification == 0) {   // if true stress/strain graph is chosen by user
    // calculate true stress
    true_stress[count] = eng_stress[count] * ( 1 + eng_strain[count]);
    // calculate true strain
    double number = 1 + eng_strain[count];
    double logOfNumber = log10(number);
    true_strain[count] = double logOfNumber;
  }

  // TODO: update counters on touchscreen


  if (count == STEPS) { // we have reached the end of our experiment

    // TODO: display message and final results on touchscreen
    while(true); // infinite loop, no more code execution
  }

  // end when load values begin to decrease drastically, possibly over a few measurements
  if (count > 10){
   float sum_of_5 = 0;

   for (int i = 0; i <= 4; i++){ // loop creates i variable + runs for when i<=4
    sum_of_5 += load[count - i];
   }

   float average_of_5 = sum_of_5 / 5;

   if (load[count] < average_of_5){
    // TODO: display message and final results on touchscreen
    while(true); // infinite loop, no more code execution
   }

  }
  if (deflection[count] >= machine_h - 0.5){
    // we have reached deflection limit
    // TODO: display message and final results on touchscreen
    while(true); // infinite loop, no more code execution
  }
  // TODO: close file
  count++;
  delay(500); // readings taken every 500ms so delay loop
}
```

References

[1] Weinburg. 2018. Planetary Gears – a masterclass for mechanical engineers. [online] Available at: <https://www.engineeringclicks.com/planetary-gears/> [Accessed 2 May 2022].

[2] Econation. 2022. Wood, Sustainable wood, Sustainable uses of wood. [online] Available at: <https://econation.one/wood/> [Accessed 3 June 2022].

[3] Small99. 2022. Is polypropylene sustainable? -Small99 Materials & Production. [online] Available at: <https://small99.co.uk/materials/is-polypropylene-sustainable/> [Accessed 3 June 2022].

[4] Wiley. 2020. Discover the Sustainable Characteristics of Metal.[online] Available at: <https://www.wiley.com/network/professionals/sustainability/discover-the-sustainable-characteristics-of-metal> [Accessed 3 June 2022].